RKNN3 Toolkit Lite Python API Reference

Document ID: RK-KF-YF-432

Version: V1.2.0

Date: 2026-10-23

Confidentiality: □Top Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY.

THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip.

All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is the RKNN3 Toolkit Lite Python API Reference Manual.

Target Audience

This document is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

Version	Author	Date	Description	Reviewer
V0.5.0	HPC Team	2025-12-05	Initial release	Vincent
V1.0.0	HPC Team	2026-01-05	Improved API descriptions	Vincent
V1.0.4	HPC Team	2026-05-06	1. Added `input_callback`; 2. Added LLM inference pause and resume	Vincent
V1.1.0	HPC Team	2026-08-06	1. Added encrypted-model loading interface description; 2. Added multi-Session interface descriptions; 3. Added dynamic Shape query interface description; 4. Added LoRA / KV Cache interface descriptions; 5. Added asynchronous session-control interface descriptions	Vincent
V1.2.0	HPC Team	2026-10-23	1. Added API descriptions for paged KV Cache and Grammar-constrained generation; 2. Expanded the `source` parameter description for `load_rknn` and the `keep_grammar` inference parameter description	Vincent

Table of Contents

1. API Reference Overview

   1.1 Applicable Versions and Platforms

   1.2 Installation and Import

   1.3 Basic Usage Flow

      General Models

      LLM/VLM

   1.4 Mapping Between Python Types and C Types

   1.5 Exception-Handling Conventions

2. RKNN3Lite Programming Model

   2.1 Object Lifecycle

   2.2 Relationship Among Model, Runtime, and Session

2.3 Synchronous and Asynchronous Execution

General Models

LLM/VLM

2.4 Memory Ownership and Release

High-Level Input and Output

Tensor Memory Created by Runtime

User-Provided Model Runtime Memory

Output Callback Memory

2.5 Thread and Callback Lifetime

3. RKNN3Lite Class

3.1 Object Initialization and Release

init

release

3.2 Model Loading and Runtime Initialization

load_rknn

init_runtime

3.3 General-Model Inference

inference

3.4 Asynchronous Inference and Waiting

inference_async

wait

get_outputs

3.5 Tensor Attributes and Dynamic Shape

get_inputs_tensor_attr

get_dynamic_shape_infos

get_outputs_tensor_attr

set_shape

3.6 Tensor and Memory Management

create_mem

create_mem_from_phys

create_mem_from_fd

destroy_mem

mem_sync
mem_sync_range
set_internal_mem
set_weight_mem
set_input_name_alias

3.7 LLM Session Lifecycle

init_llm_session
get_session_count

3.8 LLM Inference Control

session_run
session_run_async
session_stop
session_pause
session_resume
session_query_state
set_llm_param

3.9 Chat Template, Function Calling, and Grammar

set_chat_template
set_function_tools
init_grammar
set_grammar
clear_grammar

3.10 KVCache Management

set_kvcache_policy
load_kvcache
save_kvcache
clear_kvcache
set_kvcache_mem
set_kvcache_group
set_kvcache_pages

3.11 LoRA Management

init_lora
load_lora
unload_lora
enable_lora

disable_lora
query_lora
3.12 Callback Configuration
Callback Configuration Example
create_output_tensors
3.13 Profile and Debugging
dump_features
profile_ops
profile_mem
3.14 Image Memory and Image Processing
im_mem_create
im_mem_destroy
im_cvt_color
3.15 Device Query and Context Duplication
get_sdk_version
get_devices_id
dup_context
rknn3_query
4. RKNN3 Type Definitions
4.1 Constants, Status Codes, and Enumerations
4.1.1 Constant Definitions
4.1.2 Status Codes
4.1.3 RKNN3QueryCmd
4.1.4 RKNN3TensorType
4.1.5 RKNN3TensorQntType
4.1.6 RKNN3TensorLayout
4.1.7 RKNN3MemAllocFlags
4.1.8 RKNN3MemSyncMode
4.1.9 RKNN3CoreMask
4.1.10 RKNN3KVCachePolicy
4.1.11 RKNN3KVCacheClearPolicy
4.1.12 RKNN3LLMInputType
4.1.13 LLMCallState
4.1.14 RKNN3GrammarType

4.1.15 Other Enumeration Types

4.2 General Structures

4.2.1 RKNN3InputOutputNum

4.2.2 RKNN3PerfDetail

4.2.3 RKNN3PerfRun

4.2.4 RKNN3SDKVersion

4.2.5 RKNN3CustomString

4.2.6 RKNN3Config

4.2.7 RKNN3ShapeInfo

4.2.8 RKNN3ShapeConfig

4.2.9 RKNN3InitExtend

4.2.10 RKNN3NodeMemInfo

4.2.11 RKNN3DevMemInfo

4.2.12 RKNN3Device

4.2.13 RKNN3Devices

4.2.14 Custom-Operator Structures

4.3 Tensor and Memory Structures

4.3.1 RKNN3QuantInfo

4.3.2 RKNN3TensorAttr

4.3.3 RKNN3MemSize

4.3.4 RKNN3TensorMemory

4.3.5 RKNN3Tensor

4.3.6 RKNN3AllocationInfo

4.3.7 RKNN3CoreMemSize

4.4 LLM and VLM Structures

4.4.1 Float16

4.4.2 RKNN3VocabInfo

4.4.3 RKNN3SamplingParams

4.4.4 RKNN3LLMTensor

4.4.5 RKNN3AuxTensor

4.4.6 RKNN3LLMMultiModelTensor

4.4.7 RKNN3LLMInputUnion

4.4.8 RKNN3LLMInput

4.4.9 RKNN3LLMExtendParam

4.4.10 RKNN3LLMParam

4.4.11 RKNN3LLMInferParam

4.4.12 RKLLMResult
4.4.13 RKLLMRunState
4.4.14 RKLLMResultLastHiddenLayer
4.4.15 RKNN3LLMConfig
4.4.16 RKNN3GrammarConfig
4.4.17 RKNN3GrammarSetOptions
4.5 KVCache Structures
4.5.1 RECURRENT
4.5.2 RKNN3KVCachePolicyParam
4.5.3 RKNN3KVCacheClearPolicyParam
4.5.4 RKNN3AttentionKvCacheLens
4.5.5 RKNN3KvCacheLenGroupInfo
4.5.6 RKNN3AttentionConfig
4.5.7 RKNN3CachePageAttr
4.5.8 RKNN3KvCachePageInfo
4.5.9 RKNN3KvCachePageBinding
4.6 LoRA Structure
4.6.1 RKNN3Lora
4.7 Image, Audio, and Video Structures
4.7.1 RKNN3Image
4.7.2 RKNN3Audio
4.7.3 RKNN3Video
4.7.4 RKNN3ImRect
4.7.5 RKNN3ImMetadata
4.7.6 RKNN3ImMem
4.8 Callback Types
4.8.1 LLMResultCallback
4.8.2 LLMSamplingCallback
4.8.3 LLMGetEmbedCallback
4.8.4 LLMTokenizerCallback
4.8.5 LLMOutputCallback
4.8.6 LLMInputCallback
4.8.7 RKLLMCallback
4.8.8 LLMInputCallbackParam
4.9 Wrapper Classes

4.9.1 RKNN3AuxTensorWrapper

5. Public Utility Functions

5.1 rknn_dtype_to_numpy_dtype

5.2 rknn3_get_layout_string

5.3 rknn3_get_type_string

5.4 rknn3_get_qnt_type_string

5.5 dump_tensor_attr

5.6 get_os_platform

6. Exceptions and Error Handling

6.1 Error Returns and Logs

6.2 Runtime and Model Initialization Errors

6.3 Query, Input, and Inference Errors

6.3.1 Query Failure

6.3.2 General-Model Input Errors

6.4 LLM Session and Input Errors

6.4.1 Session Index Errors

6.4.2 Mutually Exclusive LLM Main Inputs

6.4.3 Context-Length Errors

6.5 KVCache and LoRA Errors

6.5.1 KVCache

6.5.2 LoRA

6.6 Memory-Related Errors

6.6.1 Out of Memory

6.6.2 Memory Synchronization Failure

6.6.3 External-Memory Lifetime

6.7 Callback Exceptions

6.8 Asynchronous-Execution Errors

General Models

LLM

6.9 Error-Diagnosis Recommendations

1. API Reference Overview

This manual describes the Python APIs of RKNN3 Toolkit Lite. It focuses on the public interfaces and data-type definitions of `RKNN3Lite`, including call sequences, parameter meanings, return values, resource lifecycles, and the relationships among Runtime, Session, Callback, and memory in LLM/VLM scenarios.

RKNN3 Toolkit Lite is used to load and run RKNN models on Rockchip NPU platforms. Model conversion is described in *RKNN3 Toolkit Python API Reference* and is outside the scope of this manual.

1.1 Applicable Versions and Platforms

The current release of this document is V1.2.0. It applies to the matching versions of the RKNN3 Toolkit Lite Python package and RKNN3 Runtime.

Currently supported platforms are as follows:

Item	Supported Range
Hardware platforms	RK1820, RK1828, RK3572
Operating systems	Debian 10 / 11 / 12 (aarch64)
Python	3.9 / 3.10 / 3.11 / 3.12
Main Python dependencies	`numpy`, `transformers`, `jinja2`

Capabilities may vary across hardware platforms, Runtime versions, or model types. If a specific API section defines additional platform restrictions or usage conditions, follow the corresponding API description.

RKNN3 Toolkit Lite depends on the RKNN3 Runtime shared libraries on the device. The Python package loads the corresponding Runtime from predefined paths. Therefore, the Python Wheel, device Runtime, and RKNN model must remain version-compatible.

1.2 Installation and Import

Install RKNN3-Toolkit Lite using `pip3 install`. The installation procedure is as follows:

- If `python3`, `pip3`, or related programs are not installed on the system, install them first through `apt-get`, for example:

```
sudo apt-get update
sudo apt-get install -y python3 python3-dev python3-pip gcc
```

Note: Some dependency modules need to compile source code. In that case, `python3-dev` and `gcc` are required. The command above installs both packages to avoid compilation failures while installing dependencies.

- Install the dependency modules `opencv-python` and `numpy`, for example:

```
sudo apt-get install -y python3-opencv
sudo apt-get install -y python3-numpy
```

or

```
pip3 install opencv-python
pip3 install numpy
```

Notes:

1. RKNN3-Toolkit Lite itself does not depend on `opencv-python`, but the examples use this module for image processing.
2. Installing `numpy` through `pip3` may fail on Debian 10 firmware. In that case, it is recommended to install it using the `apt-get` method above.

3. Install RKNN3-Toolkit Lite

Installation packages for all platforms are located in the SDK `rknn3-toolkit-lite/packages` directory. Enter this directory and run the following command to install RKNN3-Toolkit Lite:

```
# Python 3.9
pip3 install rknn3_toolkit_lite-x.y.z-cp39-cp39-linux_aarch64.whl
# Python 3.10
pip3 install rknn3_toolkit_lite-x.y.z-cp310-cp310-linux_aarch64.whl
# Python 3.11
pip3 install rknn3_toolkit_lite-x.y.z-cp311-cp311-linux_aarch64.whl
# Python 3.12
pip3 install rknn3_toolkit_lite-x.y.z-cp312-cp312-linux_aarch64.whl
```

`RKNN3Lite` and commonly used LLM/VLM types can be imported from `rknn3lite.api`:

```
from rknn3lite.api import (
    RKNN3Lite,
    RKLLMCallback,
    RKNN3Image,
    RKNN3AuxTensorWrapper,
)
```

Enumerations, structures, and low-level type definitions can be imported from `rknn3_types`:

```
from rknn3lite.api.rknn3_types import (
    RKNN3QueryCmd,
    RKNN3TensorType,
    RKNN3TensorLayout,
    RKNN3KVCachePolicy,
    RKNN3KVCacheClearPolicy,
    RKNN3KVCacheClearPolicyParam,
)
```

For general models:

```
rknn = RKNN3Lite()
```

For LLM/VLM models:

```
rknn = RKNN3Lite(llm_mode=True)
```

`llm_mode=True` indicates that the object will initialize code related to LLM Sessions. It does not immediately create a Runtime or Session when the `RKNN3Lite` object is constructed. The underlying resources are still created later by `init_runtime()` and `init_llm_session()`.

1.3 Basic Usage Flow

The RKNN3 Toolkit Lite call flow is divided into general-model and LLM/VLM flows according to model type.

General Models

The basic synchronous inference flow is:

```
RKNN3Lite()
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
inference()
    ↓
release()
```

Basic code:

```python
from rknn3lite.api import RKNN3Lite

rknn = RKNN3Lite()

ret = rknn.load_rknn("model.rknn", "model.weight")
if ret != 0:
    raise RuntimeError("load_rknn failed")

ret = rknn.init_runtime(target="rk1820", core_mask=0x01)
if ret != 0:
    raise RuntimeError("init_runtime failed")

outputs = rknn.inference(inputs=[input_data])
if outputs is None:
    raise RuntimeError("inference failed")

rknn.release()
```

General models also support asynchronous execution:

```
inference_async()
    ↓
wait()
    ↓
get_outputs()
```

LLM/VLM

For LLM/VLM, the recommended approach is a staged flow that first initializes the Runtime, then queries model configuration, and finally initializes the Session:

```
RKNN3Lite(llm_mode=True)
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
rknn3_query(RKNN3_QUERY_LLM_CONFIG)
    ↓
Construct llm_args and RKLLMCallback
    ↓
init_llm_session()
    ↓
session_run() / session_run_async()
    ↓
release()
```

With staged initialization, the application can read the actual model values for `vocab_size`, `embedding_dim`, `max_ctx_len`, `max_position_embeddings`, KVCache configuration, and task type before creating the Session, avoiding repeated hard-coded model parameters in the application.

If the application has already determined `llm_args` and the Callback in advance, the relevant parameters can also be passed directly to `init_runtime()`, allowing the initialization flow to create the first LLM Session.

One `RKNN3Lite` LLM object can create multiple Sessions on the same Runtime. When adding Sessions, use continuous `session_index` values. Each Session can independently maintain conversation state, KVCache, and Session-specific configuration.

1.4 Mapping Between Python Types and C Types

RKNN3 Toolkit Lite uses `ctypes` to wrap enumerations, structures, pointers, and callback functions from the RKNN3 C API. The Python API further converts some low-level objects into native Python types or NumPy arrays.

Common type mappings are shown below:

Python Representation	C Representation	Description
`int`	`int32_t`, `uint32_t`, `uint64_t`, etc.	Actual bit width depends on the specific parameter or structure field
`bool`	`bool` or integer flag	Actual storage type depends on the corresponding C field

Python Representation	C Representation	Description
`str`	`const char *` / `char *`	The Python wrapper handles string encoding and transfer
`bytes` / `bytearray`	`void *` / `const void *` + size	Commonly used for binary data such as decryption data, LoRA, and KVCache
`numpy.ndarray`	Tensor data buffer	High-level inference interfaces convert or bind NumPy data to Runtime Tensors
`IntEnum` subclass	C `enum` / integer enumeration value	Such as `RKNN3QueryCmd`, `RKNN3TensorType`
`ctypes.Structure` subclass	C `struct`	Such as `RKNN3TensorAttr`, `RKNN3TensorMemory`, `RKNN3LLMConfig`
`ctypes.POINTER(T)`	`T *`	Used for Tensor, memory, arrays, and other low-level pointers
`ctypes.c_void_p`	`void *`	Commonly used for userdata, Session, or generic memory addresses
`ctypes.CFUNCTYPE` callback object	C function pointer	Used for LLM Result, Tokenizer, Embedding, Input, Output, and other callbacks
`RKNN3TensorAttr`	`rknn3_tensor_attr`	Describes Tensor Shape, dtype, Layout, quantization, and alignment information
`RKNN3TensorMemory`	`rknn3_tensor_mem`	Describes Tensor virtual address, physical address, FD, size, and other memory information
`RKNN3Tensor`	`rknn3_tensor`	Associates Tensor attributes and actual buffers through `attr` and `mem`
`RKNN3Lora`	`rknn3_lora`	LoRA Adapter description information
`RKLLMCallback`	LLM Callback structure	Stores callback functions, userdata, and Tensor configuration for Input/Output Callback

At the Python layer, `rknn3_context` is represented by an unsigned integer according to the runtime architecture, while `rknn3_session` is represented by `c_void_p`. General applications do not need to directly manipulate these two low-level types and normally manage them through `RKNN3Lite` and `session_index`.

Note that high-level Python parameters do not always map one-to-one to C parameters. For example:

- Python `session_index` is an index used by the Python wrapper to manage multiple low-level Sessions. It is not the C Session pointer itself;

- `numpy.ndarray` is an application-side data object. The high-level interface performs the required data preparation according to Tensor attributes;
- `list`, Python dictionaries, and other objects may be converted into structure arrays or corresponding C parameters before entering the Runtime.

See Chapter 4 for complete enumeration and structure definitions.

1.5 Exception-Handling Conventions

The RKNN3 Toolkit Lite Python wrapper mainly handles Runtime errors by logging error messages and indicating failure through return values. Callers should not rely only on Python `try/except`; instead, they should check return values according to each API description.

Common return conventions are shown below:

API Type	On Success	On Failure
Status API	`0`	`-1` or another non-zero / negative error code
Object or query API	Corresponding object or query result	`None`
List API	Data list	`None` or an empty result, as specified by the API
Composite return API	`(ret, data)`, `(data, count)`, etc.	Failure tuple, as specified by the API

Underlying Runtime return codes include invalid arguments, model errors, Context errors, execution failures, out-of-memory conditions, timeouts, unavailable devices, incompatible versions, and more. The Python layer defines `rknn3_runtime_api_ret_code` to convert the corresponding error codes into readable status names. See 4.1.2 Status Codes for the complete list.

Recommended handling:

```
ret = rknn.init_runtime(target="rk1820", core_mask=0x01)
if ret != 0:
    # Handle the error based on logs and the return code
    rknn.release()
    raise RuntimeError(f"init_runtime failed, ret={ret}")
```

For APIs that return objects, explicitly check for `None`:

```
llm_config = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_LLM_CONFIG)
if llm_config is None:
    raise RuntimeError("query LLM config failed")
```

Python parameter processing, file access, NumPy operations, Tokenizer logic, user-defined Callbacks, and other code outside the RKNN3 Runtime wrapper may still raise standard Python exceptions. If Python logic is executed inside a Callback, it is recommended to catch exceptions within the Callback itself and return according to the callback contract, preventing exceptions from crossing the `ctypes` callback boundary.

2. RKNN3Lite Programming Model

`RKNN3Lite` is the main Python entry point for RKNN3 Toolkit Lite. Understanding the relationships among objects, Runtime, models, Sessions, memory, and Callbacks helps ensure correct use of the APIs described in Chapter 3.

2.1 Object Lifecycle

Creating an `RKNN3Lite` Python object only creates the Python-side wrapper object and does not immediately create an RKNN3 Runtime.

The complete lifecycle for a general model is:

```
Create RKNN3Lite object
        ↓
load_rknn()
        ↓
init_runtime()
        ↓
Run inference / query / profile and other operations
        ↓
release()
```

The complete lifecycle for LLM/VLM is:

```
Create RKNN3Lite(llm_mode=True)
        ↓
load_rknn()
        ↓
init_runtime()
        ↓
init_llm_session()
        ↓
Run session_run / KVCache / LoRA / Session control and other operations
        ↓
release(session_index=...) or release()
```

Where:

- `load_rknn()` records and validates the sources of the RKNN model and weight data;
- `init_runtime()` creates the underlying RKNN3 Context and completes model loading and model initialization;
- `init_llm_session()` creates a Session on an already initialized LLM Runtime;
- `release(session_index=n)` releases only the specified LLM Session while the shared Runtime remains valid;

- `release()` without a Session argument releases all Sessions and the Runtime managed by the current object.

After resources are released, Tensors, Memory, Callback data, and low-level handles associated with that Session or Runtime must no longer be used.

To ensure that resources are also released on error paths, the application should call `release()` on exit paths. Tensor Memory created explicitly must also be released with `destroy_mem()` according to the corresponding API requirements.

2.2 Relationship Among Model, Runtime, and Session

Within an `RKNN3Lite` instance, the core objects can be understood as the following hierarchy:

```
RKNN3Lite Python object
        │
        ├── RKNN model file / Weight file
        │
        ▼
RKNN3 Runtime / Context
        │
        ├── General model: execute inference() directly
        │
        └── LLM/VLM:
              ├── Session 0
              ├── Session 1
              ├── Session 2
              └── ...
```

`load_rknn()` itself does not complete low-level model initialization. Actual model loading, Context creation, and NPU Runtime initialization occur in `init_runtime()`.

For general models, after Runtime initialization, the application can directly run `inference()`, `inference_async()`, Tensor Query, dynamic Shape, Profile, and explicit memory-management operations without creating a Session.

For LLM/VLM, Sessions are built on an already initialized Runtime/Context. Multiple Sessions share the same underlying model and Runtime, but each Session has independent conversation state. Chat Template, KVCache, inference state, and Session-specific LoRA operations are targeted using `session_index`.

When initializing multiple Sessions, `session_index` values must be created continuously. For example, if Session 0 already exists, Session 1 can be initialized next, but Session 2 cannot be created directly before Session 1.

LLM Sessions can be initialized in two ways:

Direct initialization

```
load_rknn()
    ↓
init_runtime(llm_args=..., llm_callback=...)
    ↓
Session 0 is available
```

Staged initialization

```
load_rknn()
    ↓
init_runtime()
    ↓
rknn3_query()
    ↓
init_llm_session()
```

Staged initialization is suitable when model configuration must first be queried and then `llm_args`, Embedding, and Callback are constructed according to the model's actual parameters. It also provides a clearer workflow for multi-Session management.

2.3 Synchronous and Asynchronous Execution

RKNN3 Toolkit Lite provides synchronous and asynchronous interfaces for general models and LLM Sessions, but the completion mechanisms of the two asynchronous flows are different.

General Models

Synchronous call:

```
inference()
    ↓
When the method returns, the inference has completed
```

Asynchronous call:

```
inference_async()
    ↓
wait()
    ↓
get_outputs()
```

`inference_async()` submits the task; `wait()` waits for the asynchronous task to complete; after completion, use `get_outputs()` to obtain the result.

LLM/VLM

Synchronous call:

```
session_run()
    ↓
Returns after generation completes, stops, or an error occurs
```

Asynchronous call:

```
session_run_async()
    ↓
Runtime continues execution
    ↓
LLM Callback receives output and runtime state
```

LLM asynchronous execution does not use the general-model `wait()`. The application should determine whether an asynchronous request has ended based on terminal states received by the result Callback. Common terminal states include:

- `RKLLM_RUN_FINISH`
- `RKLLM_RUN_STOP`
- `RKLLM_RUN_MAX_NEW_TOKEN_REACHED`
- `RKLLM_RUN_ERROR`

During execution, `session_query_state()` can also be used to query token counts, KVCache status, and other runtime information, and the application can call as needed:

- `session_stop()`
- `session_pause()`
- `session_resume()`

`session_run_async()` indicates asynchronous submission and callback-driven execution. Submitting multiple asynchronous requests continuously on the same Session should not be treated as equivalent to true multi-request parallelism. For task isolation or concurrency management, use multiple Sessions together with application-level scheduling and synchronization.

2.4 Memory Ownership and Release

RKNN3 Toolkit Lite supports both high-level NumPy input and explicit Tensor Memory. Memory ownership differs depending on which API creates or receives the memory.

High-Level Input and Output

High-level APIs such as `inference()` and `session_run()` usually receive Python objects such as `numpy.ndarray`, Prompt, Token, or Embed. During synchronous calls, these input objects must remain valid.

For asynchronous calls, to avoid Python buffers being released before the low-level task completes, the input data and its references must remain valid at least until the corresponding asynchronous task ends.

In VLM scenarios, if the address of NumPy data is directly converted into a `Float16 *`, `void *`, or similar pointer and written into `RKNN3Image` or another structure, the lifetime of the corresponding NumPy array must cover the entire period during which the low-level pointer is used.

Tensor Memory Created by Runtime

Runtime Tensor Memory created through the following interfaces:

- `create_mem()`
- `create_mem_from_phys()`
- `create_mem_from_fd()`

must be released through `destroy_mem()` when no longer needed, to release the corresponding RKNN3 memory object or handle.

For `create_mem()`, the actual Tensor Buffer is created and managed by the RKNN3 Runtime. This API supports both allocation by `RKNN3TensorAttr` and raw memory allocation by `size` + `core_id`. Both forms must be released through `destroy_mem()`.

For `create_mem_from_phys()` and `create_mem_from_fd()`, the physical memory, FD, or external buffer is supplied by the application. The application must ensure that the external storage remains valid while RKNN3 is using it. `destroy_mem()` releases RKNN3's association object for that memory; it does not mean that the application may destroy the external storage prematurely.

User-Provided Model Runtime Memory

The following APIs allow the application to provide memory for specific Runtime purposes:

- `set_internal_mem()`
- `set_weight_mem()`
- `set_kvcache_mem()`
- `set_kvcache_pages()`

Such memory is prepared by the application, and its lifetime must cover the period during which the Runtime or corresponding Session uses it.

In particular, `set_kvcache_pages()` binds page tables. The caller must retain all successfully bound pages until they are unbound by setting `n_pages = 0`. Do not call `destroy_mem()` on a bound page before unbinding it.

Output Callback Memory

When using LLM Output Callback, the application can create an `RKNN3Tensor` array and call `create_mem()` for each output. These Tensors are registered with the Session through:

```
RKLLMCallback.output_tensors
RKLLMCallback.n_output_tensors
```

After data is read in the callback, memory that may still be used by later Callbacks must not be released immediately. After the Session is no longer using it, call `destroy_mem()` for each Tensor.

After `release()` is called, low-level memory handles, Tensor pointers, and Session resources that depend on the Runtime/Session are considered invalid.

2.5 Thread and Callback Lifetime

RKNN3 Toolkit Lite Callbacks are connected to the low-level Runtime through `ctypes`. The low-level Runtime may invoke Python Callbacks during inference execution, so the lifetime of Callback Python objects must be explicitly maintained by the application.

Objects that usually need to remain valid for the lifetime of a Session include:

- `LLMResultCallback`
- `LLMSamplingCallback`
- `LLMTokenizerCallback`
- `LLMGetEmbedCallback`
- `LLMInputCallback`
- `LLMOutputCallback`
- `RKLLMCallback`
- userdata for each Callback
- `ctypes.py_object`
- `input_tensors_index` arrays
- `output_tensors` arrays
- NumPy, mmap, or other external buffers directly referenced by Callbacks

Do not create Callback objects or userdata only in a local scope and then lose their Python references while the Session is still running. A safer approach is to store these objects in long-lived variables, members of business objects, or a dedicated keepalive container.

Tensor pointers and `LLMInputCallbackParam` received by `LLMInputCallback` are valid only during the current callback. The application may read tokens, positions, and Tensor attributes and write to Runtime-provided target buffers within the callback, but must not retain these low-level pointers and access them after the callback returns.

If output Tensors in `LLMOutputCallback` come from application-created `output_tensors`, their underlying memory must remain valid until the final Callback that may use the output has completed.

When Python code executes inside a Callback, it is recommended to catch exceptions within the Callback and return according to the callback contract, preventing unhandled Python exceptions from crossing the `ctypes` callback boundary.

The current Python wrapper and public interfaces do not provide a unified thread-safety guarantee for all `RKNN3Lite` methods. Therefore, applications should not assume that arbitrary APIs on the same object can always be called concurrently without restrictions.

In particular, distinguish the following:

- `session_stop()`, `session_pause()`, `session_resume()`, and similar APIs are designed to change Session state while the Session is running;
- This does not mean that all APIs on the same Session are guaranteed to be thread-safe when called concurrently;
- Running, stopping, pausing, resuming, and releasing the same Session should be coordinated by the application;
- For concurrency isolation, use multiple Sessions or independent Contexts and perform synchronization at the application layer.

3. RKNN3Lite Class

RKNN3Lite is the main class of RKNN3 Toolkit Lite. It provides model loading, initialization, inference, LoRA, KVCache management, memory management, asynchronous inference, debugging, query, and other functions.

3.1 Object Initialization and Release

This section controls the lifecycle of the Python object and the underlying Runtime/Session. Create `RKNN3Lite()` for traditional models and `RKNN3Lite(llm_mode=True)` for LLM/VLM. In multi-Session scenarios, `release(session_index=...)` can be used to release only a specified Session, and finally `release()` without arguments can be called to release all Sessions and the shared Runtime.

init

API	`init`
Function	Initializes an RKNN3Lite object. Used to create an RKNN3Lite instance and configure the log level and log file path.
Parameters	`llm_mode`: Boolean. Whether to use LLM mode. Default: False.
	`verbose`: Boolean. Whether to print detailed logs to the terminal. Default: False.
	`verbose_file`: String. Log-file path. If specified, logs are also written to this file. Default: None.
Return value	None

Example:

```
from rknn3lite.api.rknn3_lite import RKNN3Lite

# Print detailed logs to the screen and also write them to inference.log
rknn_lite = RKNN3Lite(verbose=True, verbose_file='./inference.log')

# Initialize a traditional model and print detailed logs only to the screen
rknn_lite = RKNN3Lite(verbose=True)

# Initialize an LLM model
rknn_lite = RKNN3Lite(llm_mode=True, verbose=True)
```

release

API	`release`
Function	Releases RKNN resources. If `session_index` is specified, only the corresponding LLM Session is released and the shared RKNN Runtime is retained. If `session_index` is None, all LLM Sessions and the RKNN Runtime are released.
Parameters	`session_index`: Optional. Specifies the LLM Session index to release. Default: None.
Return value	Success: 0; Failure: -1

Example:

```
# Release all resources
ret = rknn_lite.release()

# Release only a specified LLM Session
ret = rknn_lite.release(session_index=0)
```

3.2 Model Loading and Runtime Initialization

Common initialization flows are as follows:

```
Traditional model: load_rknn -> init_runtime -> inference
Direct LLM initialization: load_rknn -> init_runtime(llm_args, llm_callback) ->
session_run
Staged LLM initialization: load_rknn -> init_runtime -> rknn3_query ->
init_llm_session -> session_run
```

For LLMs, the staged initialization flow is recommended: first obtain model information such as `vocab_size`, `max_ctx_len`, and `embedding_dim` through `RKNN3_QUERY_LLM_CONFIG`, and then construct the LLM parameters and Callback. This avoids hard-coding model-related parameters in the application.

load_rknn

API	`load_rknn`
Function	Sets and validates the sources of the RKNN model and weights for loading by the subsequent `init_runtime()` call. Supports loading from host-side file paths, in-memory data, or device-side file paths.
Parameters	`model_path`: Model source, interpreted according to `source`: - `source="path"`: Path to the host-side `.rknn` model file - `source="data"`: Model data in memory; supports `bytes`, `bytearray`, and `memoryview` - `source="device"`: Model file path on the device-side file system (e.g., eMMC)
	`weight_path`: Weight source, interpreted according to `source`: - `source="path"`: Path to the host-side `.weight` file - `source="data"`: Weight data in memory; supports `bytes`, `bytearray`, and `memoryview` - `source="device"`: Weight file path on the device-side file system; `None` indicates streaming weight-loading mode
	`source`: Source type for model and weights: `"path"` (default), `"data"`, or `"device"`: - `"path"`: Load from host-side file paths, corresponding to C API `rknn3_load_model_from_path` - `"data"`: Load from in-memory data, corresponding to C API `rknn3_load_model_from_data` - `"device"`: Load from device-side file paths, corresponding to C API `rknn3_load_model_from_device`
Return value	Success: 0; Failure: -1

Examples:

Load from host-side file paths (the default, backward-compatible mode):

```
ret = rknn_lite.load_rknn(model_path='model.rknn', weight_path='model.weight')
if ret != 0:
    print('Load model failed')
```

Load from device-side file paths (both files reside on the device; the host passes only their paths):

```
ret = rknn_lite.load_rknn(model_path='/data/model.rknn',
weight_path='/data/model.weight', source='device')
if ret != 0:
    print('Load model failed')
```

Load the model from a device-side file path with streaming weight loading:

```
ret = rknn_lite.load_rknn(model_path='/data/model.rknn', weight_path=None,
source='device')
if ret != 0:
    print('Load model failed')
```

Usage Notes:

- The RKNN model and weight file must be used as a matching pair. If the weight file has the same base name as the model, the corresponding weight path can be generated with `rknn_path.replace('.rknn', '.weight')`.
- For `source="path"`, `load_rknn()` checks whether the model and weight files exist. For `source="data"`, it validates the data types. For `source="device"`, the paths refer to the device file system, so host-side file existence is not checked.
- For `source="data"`, `bytearray` and `memoryview` are internally converted to `bytes` before being passed to the low-level interface. The data must remain valid until `init_runtime()` finishes loading it.
- For `source="device"`, `weight_path=None` enables streaming weight loading; the application must load the weight data in batches after `init_runtime()`.
- `load_rknn()` only records and validates the model/weight sources. Actual Runtime creation, model initialization, and NPU resource creation occur in `init_runtime()`.

init_runtime

API	`init_runtime`
Function	Initializes the runtime environment. This method must be called before inference.
Parameters	`target`: Specifies the coprocessor. Currently supports rk1820/rk1828/rk3572.
	`core_mask`: NPU Core bit mask. The number of supported NPU Cores varies by platform. Configure it according to the actual platform, the Core configuration used during model conversion, and the result of `RKNN3_QUERY_CORE_NUMBER`. For example, 0x0f (binary 0b00001111).
	`llm_args`: Dictionary containing LLM model configuration parameters.
	`llm_callback`: Callback functions for the LLM model, used for streaming inference.
	`device_id`: Device ID, used to distinguish multiple devices.
	`decrypt_key_path`: Path to the decryption key or license file, used to load an encrypted RKNN model.
	`decrypt_key_data`: Raw decryption-key data or byte stream used to load an encrypted RKNN model; mutually exclusive with `decrypt_key_path`.
	`postprocess_plugin_path`: Path to a post-processing plugin shared library (`.so`). Only applicable in non-LLM mode. The plugin is registered after `model_init` and before Tensor creation. After registration, input/output counts, output attributes, and dynamic Shape queries automatically use `RKNN3_QUERY_POSTPROCESS_*` commands.
Return value	Success: 0; Failure: -1

Notes:

- The currently configurable `llm_args` parameters and their meanings are as follows:
  - `top_k`: Number of highest-probability vocabulary items considered during sampling. Integer. Default depends on the model.
  - `top_p`: Nucleus-sampling probability threshold. Floating-point value in [0, 1], used to control sampling diversity.
  - `temperature`: Temperature parameter controlling randomness. Floating-point value, normally greater than 0. Smaller values make generation more deterministic.
  - `repeat_penalty`: Repetition penalty coefficient. Floating-point value used to penalize repeated generation and reduce repetitive content.
  - `frequency_penalty`: Frequency penalty. Floating-point value based on token occurrence frequency.
  - `presence_penalty`: Presence penalty. Floating-point value based on whether a token has already appeared.

- `vocab_size`: Vocabulary size. Integer representing the total size of the model vocabulary.
- `special_bos_id`: Beginning-of-Sequence token ID. Integer.
- `special_eos_id`: End-of-Sequence token ID. Integer.
- `linefeed_id`: Line-feed token ID. Integer.
- `skip_special_token`: Whether to skip special tokens. Boolean; True means skip them.
- `ignore_eos_token`: Whether to ignore the EOS token. Boolean; True means ignore it.
- `keep_history`: Whether to retain conversation history. Boolean; True means retain it.
- `max_new_tokens`: Maximum number of newly generated tokens. Integer controlling maximum generated-text length.
- `logits_name`: Name of the logits output. String used to specify the output layer name.
- `max_context_len`: Maximum context length. Integer representing the maximum context that the model can process.

- `llm_callback` mainly contains six callback functions used to handle different stages of LLM inference:

  - `LLMResultCallback`: Result callback for receiving and processing generated text results.
  - `LLMSamplingCallback`: Sampling callback for custom sampling strategies or sampling-process handling.
  - `LLMTokenizerCallback`: Tokenizer callback for tokenizing and encoding input text.
  - `LLMGetEmbedCallback`: Embedding callback for obtaining input embeddings.
  - `LLMOutputCallback`: Output callback for obtaining model output tensors.
  - `LLMInputCallback`: Input callback for filling or updating specified input tensors during inference.

  Note: The implementation and parameters of these callbacks correspond directly to the functions and parameters provided in this API manual. Users may provide custom implementations as needed.

- `device_id` can be obtained using the `get_devices_id()` function, which returns the IDs of all connected coprocessors.
- To query model information before initializing the LLM Session, leave `llm_args` and `llm_callback` empty. After querying, call `init_llm_session` separately.
- `decrypt_key_path` and `decrypt_key_data` are used for encrypted model loading and must not be set at the same time.
- `postprocess_plugin_path` is not supported in LLM mode. This parameter replaces the previously exposed standalone `register_custom_ops_plugins` method. Users should pass the plugin path during `init_runtime`.

Example for a traditional model:

```
...
# Get device id
device_id = rknn_lite.get_devices_id()
# Initialize runtime environment
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0x01,
device_id=device_id[0])
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

Example for a large language model:

```
ARGS = [{"max_new_tokens":256, "top_k":1, "repeat_penalty":1.1, "special_eos_id":
151645, ...}]
callback = RKLLMCallback()
...

ret = rknn_lite.init_runtime(target='rk1820', core_mask=0xff, llm_args=ARGS,
llm_callback=callback)
if ret != 0:
  print('Init runtime environment failed')
  exit(ret)
```

Example for an encrypted model:

```
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0x01,
decrypt_key_path='./license.key')
if ret != 0:
    print('Init encrypted model failed')
    exit(ret)
```

Example for a post-processing plugin:

```
ret = rknn_lite.init_runtime(
    target='rk1820',
    core_mask=0x01,
    postprocess_plugin_path='./librknn3_postprocess.so'
)
if ret != 0:
    print('Init runtime with postprocess plugin failed')
    exit(ret)
```

Usage Notes:

- `core_mask` should match the Core configuration used for model conversion and Runtime execution. You can query the Core count first and then construct a valid bit mask according to the available Cores.

- For an LLM, if the model configuration needs to be queried first, omit `llm_args` and `llm_callback`; after Runtime initialization, obtain the configuration through `rknn3_query(RKNN3_QUERY_LLM_CONFIG)`, then call `init_llm_session()`.
- When using a post-processing plugin, subsequent output count, output attributes, and dynamic Shape queries should use the results after plugin registration.

3.3 General-Model Inference

Before running inference on a traditional model, it is recommended to first query the input Tensor attributes to confirm the number of inputs, Shape, data type, and Layout, and then prepare the corresponding `numpy.ndarray` input.

inference

API	`inference`
Function	Runs inference for a traditional model.
Parameters	`inputs`: List of input data. Each element is a `numpy.ndarray`.
	`data_format`: List of data-format strings. Each element can be `'undefined'`, `'nhwc'`, or `'nchw'`. `'nhwc'` and `'nchw'` are valid only for 4-D inputs. For non-4-D inputs, set it to `undefined` or leave it unset. Default: None.
Return value	List of output data, each element being a `numpy.ndarray`; returns None on failure.

Example:

```
outputs = rknn_lite.inference(inputs=[input_data], data_format=['nhwc'])
```

Usage Notes:

- The number of `inputs` must match the model input count. This can be confirmed in advance through `get_inputs_tensor_attr()` or `rknn3_query(RKNN3_QUERY_IN_OUT_NUM)`.
- Construct input according to `RKNN3TensorAttr.dtype`, `layout`, and `shape`. For 4-D image inputs, `nchw` / `nhwc` can be specified explicitly; non-image or lower-dimensional inputs normally use `undefined`.

3.4 Asynchronous Inference and Waiting

Traditional-model asynchronous inference uses the following "submit—wait—retrieve results" flow:

```
inference_async -> wait -> get_outputs
```

The asynchronous interface is suitable when NPU execution needs to overlap with other CPU-side work. Before calling `get_outputs()`, first confirm that the asynchronous task has completed.

inference_async

API	**`inference_async`**
Function	Runs traditional-model inference asynchronously.
Parameters	`inputs`: List of input data.
	`data_format`: List of data formats.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.inference_async(inputs=[data])
```

wait

API	**`wait`**
Function	Waits for asynchronous inference to complete.
Parameters	None
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.wait()
```

get_outputs

API	`get_outputs`
Function	Retrieves outputs from asynchronous inference.
Parameters	None
Return value	List of output data.

Example:

```
outputs = rknn_lite.get_outputs()
```

3.5 Tensor Attributes and Dynamic Shape

For a dynamic-Shape model, `get_dynamic_shape_infos()` can first be used to obtain all supported Shape combinations, and the runtime mode can then be selected according to the actual input Shape. The high-level `inference()` can match the corresponding combination based on the input Shape. `set_shape()` is suitable when the Shape must be explicitly switched before querying attributes, running Profile, or fixing a runtime configuration.

get_inputs_tensor_attr

API	`get_inputs_tensor_attr`
Function	Obtains model input Tensor attribute information.
Parameters	None
Return value	List of input Tensor attributes; returns None on failure.

Example:

```
attrs = rknn_lite.get_inputs_tensor_attr()
```

Usage Notes: The returned attributes can be used to inspect input name, logical Shape, Layout, data type, aligned size, and associated NPU Core. Before execution, use this information to verify that preprocessing output matches model input requirements.

get_dynamic_shape_infos

API	`get_dynamic_shape_infos`
Function	Obtains the dynamic Shape combinations supported by a traditional model. This interface is only applicable in non-LLM mode. Static-Shape models return an empty list.
Parameters	None
Return value	List of dynamic Shape information; returns None on failure.

Example:

```
shape_infos = rknn_lite.get_dynamic_shape_infos()
```

Usage Notes:

- The returned result enumerates the model's supported Shape combinations and their corresponding `shape_id` values.
- For dynamic-Shape input, the actual input Shape can be matched against candidate Shapes one by one. For common 4-D image inputs, both NCHW and NHWC logical representations should be considered.
- Static-Shape models return an empty list.

get_outputs_tensor_attr

API	`get_outputs_tensor_attr`
Function	Obtains model output Tensor attribute information.
Parameters	None
Return value	List of output Tensor attributes; returns None on failure.

Example:

```
attrs = rknn_lite.get_outputs_tensor_attr()
```

Usage Notes: Output attributes can be used to confirm output count, names, Shapes, and dtypes. In asynchronous LLM Output Callback scenarios, they can also be used to preallocate memory for selected outputs through `create_output_tensors()`.

set_shape

API	`set_shape`
Function	Sets the model shape for dynamic input.
Parameters	`shape_id`: Shape ID.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.set_shape(0)
```

Usage Notes:

- `set_shape()` explicitly activates a dynamic Shape combination. The value is taken from the `shape_id` in the dynamic Shape information.
- Before performance analysis for a specified Shape, call this interface first, then reacquire input/output attributes and prepare corresponding data.
- General dynamic-Shape inference does not necessarily require manual invocation of this interface. If matching-Shape input is passed directly, the high-level inference flow can perform the corresponding selection.

3.6 Tensor and Memory Management

The interfaces in this section are intended for advanced scenarios requiring explicit control of NPU memory. Ordinary `inference()` normally does not require users to manually create Tensor memory. These APIs are used for custom Output Callback, external physical memory, FD memory, or user-managed weight/internal/KVCache memory. Memory created by RKNN3Lite should be released using the corresponding destruction interface.

In explicit-memory scenarios, the relationship among `RKNN3TensorAttr`, `RKNN3TensorMemory`, and `RKNN3Tensor` is as follows:

- `RKNN3TensorAttr`: Describes Tensor metadata such as name, Shape, data type, Layout, and aligned size;
- `RKNN3TensorMemory`: Describes the actual memory used by the Tensor, including virtual address, physical address, FD, size, and so on;
- `RKNN3Tensor`: Associates Tensor attributes and the actual data buffer through `attr` and `mem`.

LLM Output Callback is a typical use case for these interfaces. The application first queries output attributes, creates memory for `RKNN3Tensor`, registers the Tensor array with `RKLLMCallback.output_tensors`, reads the data in the callback after inference, and finally

releases explicitly created memory through `destroy_mem()`.

create_mem

API	`create_mem`
Function	Allocates Runtime memory and optionally binds it directly to a specified `RKNN3Tensor`. Supports allocation by Tensor attributes or by byte size.
Parameters	`attr`: `RKNN3TensorAttr` instance providing Tensor Shape, dtype, aligned size, and related information (used for allocation by Tensor attributes).
	`tensor`: Optional `RKNN3Tensor` instance. If provided, the created memory is associated with that Tensor, which is useful for Output Callbacks that need to retain both `attr` and `mem`.
	`size`: Number of bytes to allocate (for byte-size allocation). A `numpy.ndarray`, `bytes`, or similar object may also be passed; its `nbytes` is used automatically.
	`core_id`: Target NPU Core ID; default: `0`. Effective only for byte-size allocation.
	`flags`: Memory-allocation flags; default: `RKNN3_FLAG_MEMORY_CACHEABLE`.
Return value	Returns an `RKNN3TensorMemory` instance on success, or `None` on failure.

Example:

```
# Form 1: Allocate by Tensor attributes (memory only)
mem = rknn_lite.create_mem(attr)

# Form 1: Allocate and bind memory for an output Tensor
output_tensor = RKNN3Tensor()
mem = rknn_lite.create_mem(attr, output_tensor)

# Form 2: Allocate by byte size (without Tensor binding)
mem = rknn_lite.create_mem(size=3145728, core_id=0)

# Form 2: Allocate according to a NumPy array's nbytes
mem = rknn_lite.create_mem(np.zeros(1024, np.float16))
```

Usage Notes:

- Ordinary `inference()` normally does not require this API.
- In an LLM Output Callback scenario, first query output attributes through `RKNN3_QUERY_OUTPUT_ATTR`, then call `create_mem(attr, tensor)` to allocate buffers for output Tensors.
- Allocation by byte size is useful for memory not bound to a model Tensor, such as an individual page of a paged KV Cache. See `set_kvcache_pages()` in Section 3.10.

- If `tensor` is not `None`, `attr` must be an `RKNN3TensorAttr` instance; otherwise, the API returns `None`.
- Explicitly created memory should not rely solely on Python object reclamation and should be released using `destroy_mem()` after use.

Compatibility Note: `flags` is at the end of the parameter list. For backward compatibility, `create_mem(attr, tensor, flags)` remains supported. If `flags` is passed as the third positional argument, it is recognized as allocation flags rather than interpreted as a byte size.

create_mem_from_phys

API	`create_mem_from_phys`
Function	Creates Tensor memory from a physical address.
Parameters	`phys_addr`: Physical address.
	`virt_addr`: Virtual address.
	`size`: Size.
Return value	`RKNN3TensorMemory` instance, or None.

Example:

```
mem = rknn_lite.create_mem_from_phys(phys_addr, virt_addr, size)
```

create_mem_from_fd

API	`create_mem_from_fd`
Function	Creates Tensor memory from a file descriptor.
Parameters	`fd`: File descriptor.
	`virt_addr`: Virtual address.
	`size`: Size.
	`offset`: Offset.
Return value	`RKNN3TensorMemory`, or None.

Example:

```
mem = rknn_lite.create_mem_from_fd(fd, virt_addr, size)
```

destroy_mem

API	`destroy_mem`
Function	Destroys Tensor memory.
Parameters	`mem`: `RKNN3TensorMemory` instance.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.destroy_mem(mem)
```

Usage Notes: When using Output Callback, output memory can be created through `create_output_tensors()`. Before exit, iterate over `output_tensors[i].mem` and call `destroy_mem()`. Therefore, output memory created explicitly by the application should be managed as a paired lifecycle.

mem_sync

API	`mem_sync`
Function	Synchronizes the entire Tensor memory so that CPU-side and device-side data remain consistent.
Parameters	`mem`: `RKNN3TensorMemory`.
	`sync_mode`: `RKNN3MemSyncMode` enumeration value.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.mem_sync(mem, sync_mode)
```

mem_sync_range

API	`mem_sync_range`
Function	Synchronizes a specified byte range within Tensor memory, suitable when only part of a buffer is updated.
Parameters	`mem`: `RKNN3TensorMemory`.
	`sync_mode`: `RKNN3MemSyncMode` enumeration value.
	`offset`: Byte offset starting from `mem.virt_addr`.
	`length`: Number of bytes to synchronize. Must satisfy `offset + length <= mem.size`.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.mem_sync_range(mem, sync_mode, offset=0, length=4096)
```

set_internal_mem

API	`set_internal_mem`
Function	Sets user-allocated internal memory for multiple NPU Cores. Each memory object in the list identifies the target Core through its `core_id`.
Parameters	`mem_list`: List of `RKNN3TensorMemory` pointers.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.set_internal_mem(mem_list)
```

set_weight_mem

API	`set_weight_mem`
Function	Sets user-allocated weight memory for multiple NPU Cores. The current version supports RK3572 only.
Parameters	`mem_list`: List of `RKNN3TensorMemory` pointers. The `core_id` of each object identifies the target Core.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.set_weight_mem(mem_list)
```

set_input_name_alias

API	`set_input_name_alias`
Function	Establishes an alias mapping between the actual model input name and the input name expected by Runtime/Session. Used when a model input name differs from the fixed Session input name.
Parameters	`src_input_name`: Actual input name used by the model, for example `input_embedding`.
	`dst_input_name`: Input name expected by Runtime/Session, for example `input_embeds`.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.set_input_name_alias('input_embedding', 'input_embeds')
```

3.7 LLM Session Lifecycle

A single Runtime can manage multiple LLM Sessions. It is recommended to create Sessions using continuous `session_index` values and configure Chat Template, KVCache policy, and inference parameters separately for each Session. Different Sessions can share the same model Runtime while maintaining their own conversation state and KVCache.

init_llm_session

API	`init_llm_session`
Function	Initializes an LLM Session separately. In some scenarios, users may want to query model information before initializing an LLM Session. In that case, `init_llm_session` can be called separately. This API can also be used to initialize additional LLM Sessions in sequence.
Parameters	`llm_args`: LLM configuration-parameter dictionary or list of configuration-parameter dictionaries.
	`llm_callback`: Callback function, which can be a dict or `RKLLMCallback` structure.
	`session_index`: LLM Session index. Default: 0. When initializing an additional Session, specify the index continuously according to the current number of Sessions.
Return value	Success: 0; Failure: -1

Note:

The recommended workflow is:

1. `load_rknn(model_path, weight_path)`
2. `init_runtime(target, core_mask)` (do not pass `llm_args` or `llm_callback`; at this point only `rknn3_init` + model loading + model initialization are performed)
3. `rknn3_query(...)` (query model information such as I/O, Core count, memory size, etc.)
4. `init_llm_session(llm_args, callback)` (perform `rknn3_session_init` + callback registration)
5. `session_run(...)` (run inference)

Example:

```
ret = rknn_lite.init_runtime(target='rk1820', core_mask=LLM_CORE_MASK)

... # Execute rknn3_query

ret = rknn_lite.init_llm_session(llm_args=ARGS, llm_callback=callback)
```

Multi-Session initialization example:

```
ret = rknn_lite.init_llm_session(llm_args=ARGS, llm_callback=callback,
session_index=1)
```

Usage Notes:

- After completing `RKNN3_QUERY_LLM_CONFIG`, it is recommended to construct `llm_args` using actual information such as `vocab_size` and `max_ctx_len`.
- In multi-Session scenarios, create Sessions using continuous `session_index` values such as 0, 1, and 2. Each Session can register different Callbacks.
- After Session initialization, Chat Template and KVCache policy are usually configured before calling `session_run()`.

get_session_count

API	`get_session_count`
Function	Gets the number of currently initialized LLM Sessions.
Parameters	None
Return value	Returns the Session count on success; returns -1 on failure.

Example:

```
session_count = rknn_lite.get_session_count()
```

Usage Notes: This API gets the number of currently initialized LLM Sessions and can help the application allocate a new `session_index` or inspect resources. After releasing an individual Session, the application should reconfirm the currently valid Session range.

3.8 LLM Inference Control

LLM inference supports both synchronous execution and asynchronous submission with runtime control. Synchronous `session_run()` returns after the current inference finishes. `session_run_async()` relies on the result Callback to determine actual completion. `session_stop()`, `session_pause()`, and `session_resume()` can be called while inference is running and can be used together with `session_query_state()` to obtain current token, KVCache, and LoRA state.

session_run

API	`session_run`
Function	Runs LLM model inference and supports streaming output.
Parameters	`inputs`: List of input data for multimodal models, such as image or audio. The list elements can be dedicated types such as `RKNN3Image`, `RKNN3Audio`, `RKNN3Video`, or `RKNN3AuxTensor`.

API	`session_run`
	`prompt` : Text prompt used as LLM model input.
	`embeds` : Input embedding vector for scenarios that skip tokenization. Type: `numpy.ndarray` , typically a 2-D or 3-D floating-point Tensor.
	`tokens` : One-dimensional token ID array (`ndarray` or `list`), automatically converted to int32. Suitable when tokenization has already been performed and raw token IDs should be passed directly instead of a text prompt.
	`keep_history` : Whether to retain history context during inference, which may affect performance. Type: bool. Default: None, meaning use the model default setting.
	`max_new_tokens` : Maximum number of tokens generated during inference, which may affect performance. Type: int. Default: None, meaning use the model default setting.
	`prefill_only` : Whether to perform only one Prefill. Type: bool. Default: None, meaning use the Session default setting.
	`disable_sampling` : Whether to disable token sampling after a single Prefill. Type: bool. Default: None, meaning use the Session default setting.
	`keep_grammar` : Whether to keep the grammar matcher state across runs. Type: bool. Default: None, meaning use the Session default setting (false). When enabled, the per-request grammar matcher is NOT re-initialized at the start/end of each `session_run()` , retaining the grammar state for continuous constrained generation across multiple runs.
	`enable_thinking` : Whether to enable thinking mode during inference, which may affect performance. Type: bool. Default: False, meaning thinking mode is disabled.
	`session_index` : LLM Session index. Default: 0.
Return value	`ret` (0 for success, -1 for failure) and a performance-statistics list `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]` .

Note: `prompt` , `embeds` , and `tokens` are mutually exclusive. `inputs` is used for multimodal or auxiliary input and can be combined with `prompt` according to model requirements.

Example: LLM model inference example:

```
...
prompt = "Explain how an LLM model works."
ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] =
rknn_llm.session_run(prompt=prompt)
if ret != 0:
  print('RKNN llm inference failed!')
  exit(ret)
```

VLM model inference example:

```
...
# Get the output of the vision model and convert it to the required format
outputs = np.float16(np.expand_dims(rknn_vision.inference(inputs=[feature])[0], 0))

prompt = "<image>Please describe the image content"
inputs = []
llm_input = RKNN3Image()
llm_input.image_embed = outputs.ctypes.data_as(ctypes.POINTER(Float16))
llm_input.n_image_tokens = outputs.shape[1]
llm_input.n_image = outputs.shape[0]
llm_input.image_width = 392
llm_input.image_height = 392
llm_input.image_start = "<|vision_start|>".encode('utf-8')
llm_input.image_end = "<|vision_end|>".encode('utf-8')
llm_input.image_content = "<|image_pad|>".encode('utf-8')
inputs.append(llm_input)

# Run LLM inference
ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] =
rknn_llm.session_run(inputs=inputs, prompt=prompt)
if ret != 0:
  print('RKNN LLM inference failed!')
  exit(ret)
```

Usage Notes:

- `prompt`, `embeds`, and `tokens` are mutually exclusive main-input forms. `inputs` may additionally carry `RKNN3Image`, `RKNN3Audio`, `RKNN3Video`, or `RKNN3AuxTensorWrapper`.
- Boolean parameters such as `keep_history`, `prefill_only`, `disable_sampling`, and `keep_grammar` accept integer values such as `0` and `1`. The Python wrapper automatically converts them to `bool` before passing them to the Runtime; `None` means use the Session default setting.
- For independent requests or evaluation tasks, use `keep_history=False` to avoid sharing context across different inputs. For continuous conversation, use `keep_history=True` to reuse history context.
- Performance tests can pass precomputed `embeds` directly to exclude Tokenizer/Embedding lookup time from the NPU inference process.
- The returned `n_prefill_tokens`, `n_decode_tokens`, `llm_start_time`, and `llm_end_time` can be combined with the first-token callback time to calculate TTFT, Prefill TPS, and Decode TPS.

session_run_async

API	**`session_run_async`**
Function	Runs LLM model inference asynchronously.
Parameters	`inputs`: List of input data for multimodal models.
	`prompt`: Text prompt.

API	`session_run_async`
	`embeds`: Embedding vector.
	`tokens`: One-dimensional token ID array (`ndarray` or `list`), automatically converted to int32. Suitable when tokenization has already been performed and raw token IDs should be passed directly rather than text prompt input.
	`keep_history`: Whether to retain history context.
	`max_new_tokens`: Maximum number of newly generated tokens.
	`prefill_only`: Whether to perform only one Prefill. Default: None.
	`disable_sampling`: Whether to disable token sampling after a single Prefill. Default: None.
	`keep_grammar`: Whether to keep the grammar matcher state across runs. Default: None (false). When enabled, the per-request grammar matcher is NOT re-initialized at the start/end of each `session_run()`, retaining the grammar state for continuous constrained generation across multiple runs.
	`enable_thinking`: Whether to enable thinking mode.
	`session_index`: LLM Session index. Default: 0.
Return value	`ret, stats`, where `stats` has the format `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]`.

Note: `prompt`, `embeds`, and `tokens` are mutually exclusive; `inputs` can be combined with `prompt` according to model requirements.

Example:

```
ret, stats = rknn_lite.session_run_async(prompt="Async prompt")
```

Usage Notes:

- The return value only indicates that the asynchronous task has been submitted; it does not mean generation is complete. Actual completion should be determined from terminal result-Callback states such as FINISH/STOP/MAX_NEW_TOKEN_REACHED/ERROR.
- Boolean parameters such as `keep_history`, `prefill_only`, `disable_sampling`, and `keep_grammar` accept integer values such as `0` and `1`. The Python wrapper automatically converts them to `bool` before passing them to the Runtime; `None` means use the Session default setting.
- During asynchronous execution, `session_query_state()` can be called to poll token and KVCache state.
- Repeatedly calling the asynchronous API on the same Session is more suitable for submission/queuing. For truly parallel multi-request scenarios, multiple independent Sessions

are preferred.

session_stop

API	`session_stop`
Function	Stops a running LLM Session.
Parameters	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.session_stop()
```

Usage Notes: The application can call `session_stop()` from `LLMResultCallback` according to business conditions, after which the result Callback enters the `RKLLM_RUN_STOP` terminal state. This is suitable for application-defined stop words, external cancellation requests, or timeout control.

session_pause

API	`session_pause`
Function	Pauses a running LLM Session.
Parameters	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.session_pause()
```

Usage Notes: `session_pause()` can pause inference during the Prefill or Decode phase. After a successful pause, the Session state is retained and `session_resume()` can later continue the current inference rather than starting a new request.

session_resume

API	`session_resume`
Function	Resumes a paused LLM Session.
Parameters	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.session_resume()
```

Usage Notes: This API should be paired with `session_pause()`. The result Callback may receive `RKLLM_RUN_PAUSE` and `RKLLM_RUN_RESUME` states, which the application can use to update UI or task status.

session_query_state

API	`session_query_state`
Function	Queries the current state of an LLM Session.
Parameters	`session_index`: LLM Session index. Default: 0.
Return value	Returns `RKLLMRunState` on success; otherwise returns None.

Example:

```
state = rknn_lite.session_query_state()
```

Usage Notes:

- Common fields include `n_total_tokens`, `n_max_tokens`, `n_prefill_tokens`, `n_decode_tokens`, `kvcache_policy`, and `n_loras_enabled`.
- The condition `n_total_tokens >= n_max_tokens - max_new_tokens` can be used to determine whether the context is close to full, allowing the application to proactively clear cache as needed.
- In multi-Session scenarios, pass the corresponding `session_index`; the returned state applies only to that Session.

set_llm_param

API	`set_llm_param`
Function	Sets LLM parameters for an initialized Session.
Parameters	`params`: `RKNN3LLMParam` structure.
	`n_params`: Number of parameters. Default: 1.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.set_llm_param(params, 1)
```

3.9 Chat Template, Function Calling, and Grammar

Chat Template determines how the user Prompt is combined with the system prompt, user prefix, and assistant prefix before being sent to the Tokenizer. If the input Prompt is already fully formatted by the application, the template can be set to empty strings so the Prompt is passed to the model unchanged. For continuous-conversation scenarios, the corresponding model-specific system/prefix/postfix can be configured. The template can be updated at different stages of the same Session.

Grammar (constrained generation) is used to restrict the output format of an LLM, for example to fixed-field JSON, classification labels, order numbers, or Tool Calling structures. It restricts the selectable tokens during generation, so the output can be guaranteed to satisfy the specified grammar or structure. However, it does not guarantee that the output is correct at the business or factual level; the application should still perform the required business validation.

set_chat_template

API	`set_chat_template`
Function	Sets the LLM chat template.
Parameters	`system_prompt`: System prompt.
	`prompt_prefix`: Prompt prefix.
	`prompt_postfix`: Prompt postfix.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
...
system_prompt  = "<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n"
prompt_prefix  = "<|im_start|>user\n"
prompt_postfix = "<|im_end|>\n<|im_start|>assistant\n"

ret = rknn_lite.set_chat_template(system_prompt, prompt_prefix, prompt_postfix)
if ret != 0:
    print('Set chat template failed!')
    exit(ret)
```

Usage Notes:

- Standard chat models usually configure a system prompt, user prefix, and assistant postfix.
- Passing three empty strings causes the input Prompt to enter the Tokenizer unchanged. This is useful when the application has already completed Prompt formatting itself.

- Chat Template is Session configuration and can be updated dynamically according to different business stages or conversation turns.

set_function_tools

API	`set_function_tools`
Function	Sets function tools (function calling) for an LLM Session.
Parameters	`tools`: Function-tools string in JSON format.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.set_function_tools('{"tools": []}')
```

init_grammar

API	`init_grammar`
Function	Initializes the Grammar engine shared by the current RKNN Context and establishes the mapping from token IDs to token pieces. It must be called after `init_runtime()` succeeds and before the first call to `set_grammar()`. The same Context only needs to be initialized once.
Parameters	`config`: Optional `RKNN3GrammarConfig`. Used to directly pass a constructed low-level vocabulary configuration. It cannot be used together with `token_pieces`.
	`token_pieces`: Iterable sequence of token pieces arranged in token-ID order. Each element is a `str` or bytes-like object. The high-level API automatically constructs `RKNN3GrammarConfig` from it.
	`stop_tokens`: Optional list of stop-token IDs, normally set to the Tokenizer EOS token ID. Used only when the configuration is constructed automatically through `token_pieces`.
Return value	Success: 0; Failure: -1. Invalid parameter types or parameter combinations may raise a Python exception.

Example:

```
# llm_config is obtained through RKNN3_QUERY_LLM_CONFIG; the tokenizer vocabulary
must match the model.
token_pieces = (
    tokenizer.decode([token_id], skip_special_tokens=False,
                     clean_up_tokenization_spaces=False)
    for token_id in range(llm_config.vocab_size)
)
stop_tokens = ([tokenizer.eos_token_id]
               if tokenizer.eos_token_id is not None else [])

ret = rknn_lite.init_grammar(
    token_pieces=token_pieces,
    stop_tokens=stop_tokens,
)
if ret != 0:
    print('Init grammar failed!')
    exit(ret)
```

Usage Notes:

- The *i*-th item of `token_pieces` must correspond to token ID *i* in the model vocabulary. The Tokenizer, Embedding, and RKNN model must come from the same model version.
- A generator can be passed as `token_pieces`, but its contents are consumed once during initialization and cannot be reused afterward.
- If the Runtime shared library does not support Grammar-related interfaces, the call fails and returns `-1`.

set_grammar

API	`set_grammar`
Function	Sets a structured-output Grammar for the current or subsequent requests of an initialized LLM Session. `init_grammar()` and `init_llm_session()` must already have completed before this API is called.
Parameters	`grammar_str`: Grammar content of type `str`, `bytes`, or `None`. For JSON Schema, EBNF, Regex, and Structural Tag types, pass the corresponding rule text. For Builtin JSON, normally pass `None`.
	`grammar_type`: Grammar type, specified with the `RKNN3GrammarType` enumeration. If omitted, it is equivalent to `RKNN3_GRAMMAR_BUILTIN_JSON`.
	`options`: Optional `RKNN3GrammarSetOptions` configuration. Passing `None` sets no additional options. To use Runtime default initialization values, create an options object with `RKNN3GrammarSetOptions.default()`.
	`session_index`: LLM Session index. Default: 0. Grammar configuration belongs to the specified Session.
Return value	Success: 0; Failure: -1. Invalid `grammar_str` or `options` types may raise a Python exception.

Example:

```
import json
from rknn3lite.api.rknn3_types import RKNN3GrammarType

person_schema = json.dumps({
    'type': 'object',
    'properties': {
        'name': {'type': 'string'},
        'age': {'type': 'integer', 'minimum': 0, 'maximum': 150},
    },
    'required': ['name', 'age'],
    'additionalProperties': False,
})

ret = rknn_lite.set_grammar(
    person_schema,
    RKNN3GrammarType.RKNN3_GRAMMAR_JSON_SCHEMA,
)
if ret != 0:
    print('Set grammar failed!')
    exit(ret)

ret, stats = rknn_lite.session_run(
    prompt='Extract name and age from: Alice is 28 years old. Reply only with
JSON.',
    keep_history=False,
    max_new_tokens=128,
)
```

Examples of using Builtin JSON, Regex, and EBNF:

```
# Restrict output to any valid JSON
rknn_lite.set_grammar(
    None, RKNN3GrammarType.RKNN3_GRAMMAR_BUILTIN_JSON
)

# Restrict output to an order-number format
rknn_lite.set_grammar(
    r'ORD-[0-9]{6}', RKNN3GrammarType.RKNN3_GRAMMAR_REGEX
)

# Restrict output to one of several fixed labels
rknn_lite.set_grammar(
    'root ::= "positive" | "negative" | "neutral"\\n',
    RKNN3GrammarType.RKNN3_GRAMMAR_EBNF,
)
```

Usage Notes:

- Set the required rule before each `session_run()` or `session_run_async()` call. If different requests need different rules, call `clear_grammar()` before setting a new rule.
- By default the grammar matcher is re-initialized around each `session_run()`. To retain grammar matching state across multiple `session_run()` calls (for example, splitting one long response into multiple runs for continuous generation), pass `keep_grammar=True` to `session_run()`; the matcher is then not reset at the start/end of each run.
- Rule semantics for JSON Schema, Regex, EBNF, and Structural Tag are parsed by the Runtime Grammar engine. It is recommended to validate simple rules first and configure an appropriate `max_new_tokens` value for generation.
- Grammar only constrains output format. The Prompt should still clearly describe the content to generate; it is recommended to instruct the model to output only the target format.

clear_grammar

API	`clear_grammar`
Function	Clears the Grammar currently configured for the specified LLM Session, so subsequent inference is no longer constrained by that rule.
Parameters	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1.

Example:

```
ret = rknn_lite.clear_grammar()
if ret != 0:
    print('Clear grammar failed!')
    exit(ret)

# Grammar state and KVCache are independent; independent requests normally also
need to clear the context cache.
rknn_lite.clear_kvcache()
```

Usage Notes: `clear_grammar()` only clears the output constraint; it does not clear conversation history or KVCache. Multiple Sessions maintain their Grammar configurations independently, so pass the corresponding `session_index` when calling this API.

3.10 KVCache Management

KVCache APIs cover cache policy, persistence, clearing, user memory, and length grouping, as well as paged KV Cache binding. A common usage flow is:

```
init_llm_session -> set_kvcache_policy -> session_run
                                    -> save_kvcache / load_kvcache
                                    -> session_query_state -> clear_kvcache
```

KVCache can be loaded from either a file path or in-memory data. The application can combine `n_total_tokens` and `n_max_tokens` to determine whether context is approaching its upper limit.

set_kvcache_policy

API	**`set_kvcache_policy`**
Function	Sets the KVCache policy.
Parameters	`policy`: `RKNN3KVCachePolicy` enumeration value.
	`param`: `RKNN3KVCachePolicyParam` structure, or None.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.set_kvcache_policy(RKNN3KVCachePolicy.RKNN3_KVCACHE_POLICY_DEFAULT)
```

Usage Notes:

- `RKNN3_KVCACHE_POLICY_NORMAL` is suitable for regular context caching. `RKNN3_KVCACHE_POLICY_RECURRENT` is suitable for scenarios that need to cyclically reuse cache space. Select the strategy according to the model and business requirements.
- In multi-Session scenarios, KVCache policy is configured separately by `session_index`.
- If a policy requires additional parameters, pass `RKNN3KVCachePolicyParam` at the same time.

load_kvcache

API	**`load_kvcache`**
Function	Loads KVCache from a file or memory.
Parameters	`kvcache_path`: KVCache file path.
	`kvcache_data`: KVCache data in memory.
	`size`: Data size.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.load_kvcache(kvcache_path='cache.bin')
```

Usage Notes: Two sources are supported: pass `kvcache_path` directly, or read the file into memory and load it through `kvcache_data + size`. After a successful load, `session_run()` can continue execution, allowing restoration of a previously saved context-cache state.

save_kvcache

API	**`save_kvcache`**
Function	Saves KVCache to a file.
Parameters	`kvcache_path`: Save path.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.save_kvcache('cache.bin')
```

Usage Notes: Normally used to save the current Session KVCache after a Prefill/Decode sequence. Later, `load_kvcache()` can restore it under the same model and compatible Session configuration, reducing repeated context computation.

clear_kvcache

API	`clear_kvcache`
Function	Clears KVCache.
Parameters	`clear_policy`: `RKNN3KVCacheClearPolicy` enumeration value, or `None`.
	`param`: `RKNN3KVCacheClearPolicyParam` structure. It must be provided when `clear_policy` is `RKNN3_KVCACHE_KEEP_N_TOKENS`; otherwise it can be `None`.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.clear_kvcache()

clear_param = RKNN3KVCacheClearPolicyParam()
clear_param.keep_n_tokens.n_tokens = 128
ret = rknn_lite.clear_kvcache(
    RKNN3KVCacheClearPolicy.RKNN3_KVCACHE_KEEP_N_TOKENS,
    param=clear_param,
)
```

Usage Notes:

- `RKNN3_KVCACHE_CLEAR_ALL` completely clears the current Session context and is suitable for mutually independent evaluation samples.
- `RKNN3_KVCACHE_KEEP_SYSTEM_PROMPT` preserves the cache corresponding to the system prompt while clearing KVCache, reducing repeated system-prompt computation for subsequent requests.
- `RKNN3_KVCACHE_KEEP_N_TOKENS` preserves the first `n_tokens` tokens when clearing KVCache. `RKNN3KVCacheClearPolicyParam` must be provided when this policy is used.
- Specify the correct `session_index` when clearing cache to avoid affecting other Sessions.

set_kvcache_mem

API	`set_kvcache_mem`
Function	Sets KVCache memory for specified NPU Cores.
Parameters	`mem_list`: List of `RKNN3TensorMemory`.
	`npu_core_indices`: List of Core indices.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.set_kvcache_mem(mem_list, [0,1])
```

set_kvcache_group

API	`set_kvcache_group`
Function	Sets the KV Cache length group used by the current Context. Runtime switches to the corresponding KV Cache buffer addresses and execution configuration.
Parameters	`group_id`: Group ID to activate. Must satisfy `0 <= group_id < n_groups`.
Return value	Success: 0; Failure: -1

Example:

```
infos = rknn_lite.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO
)
if infos and infos[0].n_groups > 1:
    ret = rknn_lite.set_kvcache_group(1)
```

set_kvcache_pages

API	`set_kvcache_pages`
Function	Replaces the specified paged KV Cache bindings without copying page contents. After a successful binding, the caller owns the pages and must keep them valid until unbinding.
Parameters	`bindings`: List of `RKNN3KvCachePageBinding` structures. Each entry describes the page binding for one (Core, cache type, request index) combination.
Return value	Success: 0; Failure: -1

Example:

```
from rknn3lite.api.rknn3_types import (
    RKNN3QueryCmd,
    RKNN3PageCacheType,
    RKNN3MemAllocFlags,
    RKNN3MemSyncMode,
    RKNN3KvCachePageBinding,
    RKNN3TensorMemory,
)
from ctypes import POINTER

# 1) Enumerate Cores and verify that the model uses paged KV Cache
alloc_infos = rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_ALLOCATION_INFO)
if not alloc_infos:
    raise RuntimeError('failed to query allocation info')
page_info = rknn_lite.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_KVCACHE_PAGE_INFO, index=alloc_infos[0].core_id
)
if page_info is None or page_info.n_cache_types == 0:
    raise RuntimeError('model does not use paged KV cache')

# 2) Query attention configuration, including tokens_per_page
attention_configs =
rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_ATTENTION_CONFIG)

# 3) Allocate pages by page_size and bind them
bindings = []
all_page_mems = []        # Single owner of every page; the release loop walks this
list
page_arrays = []          # Keep references to prevent the page-pointer arrays from
being reclaimed
for allocation in alloc_infos:      # Every Core binds its own page table
    info = rknn_lite.rknn3_query(
        RKNN3QueryCmd.RKNN3_QUERY_KVCACHE_PAGE_INFO, index=allocation.core_id
    )
    if info is None or info.n_cache_types == 0:
        raise RuntimeError('inconsistent paged KV cache info across cores')
    for request in range(info.max_request_num):
        for i in range(info.n_cache_types):
            cache = info.caches[i]
            page_mems = []
            for _ in range(cache.max_page_num):
                mem = rknn_lite.create_mem(
```

```
                    size=cache.page_size, core_id=info.core_id,
                    flags=RKNN3MemAllocFlags.RKNN3_FLAG_MEMORY_CACHEABLE,
                )
                rknn_lite.mem_sync(mem,
RKNN3MemSyncMode.RKNN3_MEMORY_SYNC_TO_DEVICE)
                page_mems.append(mem)
            all_page_mems.extend(page_mems)

            page_array = (POINTER(RKNN3TensorMemory) * len(page_mems))(*page_mems)
            page_arrays.append(page_array)

            binding = RKNN3KvCachePageBinding()
            binding.core_id = info.core_id
            binding.cache_type = cache.cache_type
            binding.request_index = request
            binding.n_pages = len(page_mems)
            binding.pages = page_array
            bindings.append(binding)

ret = rknn_lite.set_kvcache_pages(bindings)

# 4) Unbind with n_pages = 0 after use, then release the pages
unbinds = []
for binding in bindings:
    unbound = RKNN3KvCachePageBinding()
    unbound.core_id = binding.core_id
    unbound.cache_type = binding.cache_type
    unbound.request_index = binding.request_index
    unbound.n_pages = 0
    unbound.pages = None
    unbinds.append(unbound)
rknn_lite.set_kvcache_pages(unbinds)

for mem in all_page_mems:
    rknn_lite.destroy_mem(mem)
```

Usage Notes:

- This is a Context-level operation, not a per-Session operation. Bound pages are shared by all Sessions under the current Context. Call it only when the Context is idle.
- Use it only for paged KV Cache models. For non-paged models, `RKNN3_QUERY_KVCACHE_PAGE_INFO` returns `n_cache_types = 0`; use `set_kvcache_mem()` instead.
- The example probes `alloc_infos[0]` to decide whether paging is enabled, then iterates `alloc_infos` to bind a separate page table for every Core; a multi-Core model must not bind only the first Core.
- `n_pages = 0` unbinds the specified entry. Entries omitted from `bindings` retain their existing bindings.
- The entire shape of `bindings` is validated before applying any changes. If validation fails, the existing bindings are preserved without partial application.
- If writing the page table fails, every entry in the current batch is unbound. In either failure case, the caller may release pages allocated for that batch.

- The caller owns successfully bound pages and must keep them valid until unbinding. Unbind first using `n_pages = 0`, then call `destroy_mem()` for the pages.
- For paged models, configure internal/weight memory before binding the page tables because those memory-setting APIs may rewrite command memory.
- Paged KV Cache models do not use cache-length groups, so `set_kvcache_group()` is unnecessary.

3.11 LoRA Management

A typical LoRA lifecycle is:

```
init_lora -> query_lora -> load_lora -> enable_lora(session)
                                     -> disable_lora(session) -> unload_lora
```

The same Base Runtime can create multiple Sessions, with LoRA enabled only on specified Sessions. This allows different Sessions to run either the Base model or a LoRA model independently.

init_lora

API	`init_lora`
Function	Initializes LoRA support and loads LoRA weight data.
Parameters	`lora_weight_path`: LoRA weight file path or directory.
	`weight_data`: Optional in-memory LoRA weight data. Supports `bytes` / `bytearray` / `memoryview`, `numpy.ndarray`, or a raw pointer.
	`weight_size`: Data size in bytes when `weight_data` is a raw pointer.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.init_lora(lora_weight_path='lora_weights.bin')
```

Usage Notes: LoRA weights can be initialized from a file path or first read as bytes and then initialized from memory through `weight_data` / `weight_size`. After initialization, `query_lora()` can be called to obtain the actual adapter name and default scale.

`init_lora()` may be called repeatedly to register multiple LoRA weight files or in-memory buffers; adapter names must be unique across all registered files/buffers. In-memory buffers registered through `weight_data` are non-owning references and must remain valid until the RKNN3 Context is released.

load_lora

API	`load_lora`
Function	Loads a LoRA adapter.
Parameters	`lora`: `RKNN3Lora` structure containing `lora_name` and `scale`.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
lora = RKNN3Lora()
lora.lora_name = b'adapter1'
lora.scale = 1.0
ret = rknn_lite.load_lora(lora)
```

Usage Notes: `load_lora()` loads an initialized adapter into Runtime but does not automatically make it active for a Session. Continue by calling `enable_lora()` for the target Session.

unload_lora

API	`unload_lora`
Function	Unloads a LoRA adapter.
Parameters	`lora`: `RKNN3Lora` structure containing `lora_name` and `scale`.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.unload_lora(lora)
```

enable_lora

API	`enable_lora`
Function	Enables a LoRA adapter for inference.
Parameters	`lora`: `RKNN3Lora` structure containing `lora_name` and `scale`.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.enable_lora(lora)
```

Usage Notes: LoRA can be enabled per Session. Different Sessions can share the same Runtime while independently choosing whether to enable LoRA. `RKNN3Lora.scale` can be adjusted before enabling to control adapter strength.

Note: different adapters mapped to the same pattern slot cannot be enabled together in one Session. Different Sessions may reuse that slot; the runtime reapplies each Session's LoRA binding before inference.

disable_lora

API	`disable_lora`
Function	Disables a LoRA adapter.
Parameters	`lora`: `RKNN3Lora` structure containing `lora_name` and `scale`.
	`session_index`: LLM Session index. Default: 0.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.disable_lora(lora)
```

Usage Notes: Disables LoRA only for the specified Session. It does not unload the weights from Runtime. To fully release the adapter, call `unload_lora()` afterward.

query_lora

API	`query_lora`
Function	Queries LoRA adapter information available in the current Runtime, i.e. LoRA information initialized by `init_lora()`.
Parameters	`session_index`: LLM Session index. Default: 0.
Return value	`(lora_list, n_lora)` on success; otherwise `(None, 0)`.

Example:

```
lora_list, n_lora = rknn_lite.query_lora()
```

Usage Notes: Each element in the returned `lora_list` contains `lora_name` and `scale`. The application can query the actual list first and then select an adapter for `load_lora()` / `enable_lora()`, avoiding hard-coded adapter names.

3.12 Callback Configuration

RKNN3Lite does not provide a standalone `set_callback()` method. LLM Callbacks are registered through `init_runtime(..., llm_callback=...)` or `init_llm_session(..., llm_callback=...)`, and all callbacks are stored together in `RKLLMCallback`.

Common callbacks include:

- `result_callback`: Receives generated tokens and runtime state;
- `tokenizer_callback`: Converts input text into token IDs;
- `embed_callback`: Fills token embeddings according to token IDs;
- `input_callback`: Fills specified extended input Tensors according to current token, position, and related information during inference;
- `output_callback`: Reads model output Tensors registered by the user at specified Prefill/Decode stages.

A standard text LLM usually needs only Result, Tokenizer, and Embedding callbacks. For scenarios with additional model inputs, `input_tensors_index` and `n_input_tensors` can specify input indices handled by `input_callback`. For scenarios that need intermediate layers or output embeddings, `output_tensors` and `n_output_tensors` can register Tensors used by Output Callback.

Callbacks are invoked by the low-level Runtime during inference. Therefore, related `ctypes` callback objects, userdata, input-index arrays, and NumPy/memory objects referenced by Callbacks must remain valid throughout Session use and must not be immediately garbage-collected by Python after initialization.

Callback Configuration Example

The following example demonstrates the general Output Callback configuration relationship:

```
io_num = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_IN_OUT_NUM)

OutputTensorArray = RKNN3Tensor * io_num.n_output
output_tensors = OutputTensorArray()

for i in range(io_num.n_output):
    attr = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_OUTPUT_ATTR, index=i)
    if rknn.create_mem(attr, output_tensors[i]) is None:
        raise RuntimeError(f"create output memory failed: {i}")

callback.output_callback = LLMOutputCallback(output_callback)
callback.output_tensors = ctypes.cast(output_tensors, ctypes.POINTER(RKNN3Tensor))
callback.n_output_tensors = io_num.n_output
```

After the Session is no longer used, release the explicitly created output memory:

```
for tensor in output_tensors:
    rknn.destroy_mem(tensor.mem)
```

For Input Callback, first determine the input indices that should be filled by the application based on model input attributes, then configure `input_tensors_index` and `n_input_tensors`. Specific input names and data organization depend on the model.

create_output_tensors

API	`create_output_tensors`
Function	Creates output Tensors with allocated memory for LLM Output Callback.
Parameters	`output_indices`: List of output Tensor indices. If None, all output Tensors are used by default.
Return value	`(output_tensor_array, n_output_tensors)`; returns `(None, 0)` on failure.

Example:

```
tensors, n = rknn_lite.create_output_tensors([0,1])
```

Usage Notes:

- Mainly used with `LLMOutputCallback`. The application can also query `RKNN3_QUERY_OUTPUT_ATTR`, construct an `RKNN3Tensor` array, and call `create_mem(attr, tensor)` for each output;

- After creation, set the Tensor array to `RKLLMCallback.output_tensors` and set `n_output_tensors` accordingly;
- After the callback is triggered, use `tensor.attr` to obtain name, Shape, dtype, and related information, and use `tensor.mem.virt_addr` to access actual output data;
- These Tensors contain explicitly allocated memory. After the Session is no longer used, call `destroy_mem(output_tensors[i].mem)` for each one.

3.13 Profile and Debugging

When profiling, it is recommended to complete model initialization and one valid inference first, then call `profile_mem()` and `profile_ops()`. For a dynamic-Shape model, if a specific Shape needs to be analyzed, switch or confirm the current Shape first to avoid Profile results that do not match the expected input configuration.

dump_features

API	**`dump_features`**
Function	Dumps intermediate model features to `.npy` files.
Parameters	`dump_dir`: Dump directory.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.dump_features('./dump')
```

profile_ops

API	**`profile_ops`**
Function	Prints per-layer operation performance information.
Parameters	`log_level`: Log level.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.profile_ops(1)
```

Usage Notes: It is recommended to complete one valid `inference()` before calling this API so the Profile corresponds to an actually executed model and Shape. `log_level` can control Profile output granularity; supported values are defined by the interface.

profile_mem

API	`profile_mem`
Function	Prints memory usage for each NPU Core.
Parameters	None
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.profile_mem()
```

Usage Notes: After inference, call `profile_mem()` to inspect memory usage on each NPU Core and then call `profile_ops()` to analyze operator performance. The two APIs can be used together to locate performance bottlenecks and memory pressure.

3.14 Image Memory and Image Processing

This section covers Runtime image memory and color conversion. These interfaces differ from directly passing `RKNN3Image.image_embed` in VLM: the former process image buffers, while the latter describes multimodal embeddings that have already been extracted by a vision model.

im_mem_create

API	`im_mem_create`
Function	Creates image memory.
Parameters	`width` : Width.
	`height` : Height.
	`fmt` : Image format.
	`size` : Size.
	`core_id` : Core ID.
	`flags` : Flags.
Return value	`(ret, RKNN3ImMem)`

Example:

```
ret, im_mem = rknn_lite.im_mem_create(224, 224, fmt, 0, 0)
```

im_mem_destroy

API	`im_mem_destroy`
Function	Destroys image memory.
Parameters	`im_mem`: `RKNN3ImMem` object.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.im_mem_destroy(im_mem)
```

im_cvt_color

API	`im_cvt_color`
Function	Converts an image color space.
Parameters	`src_im_mem`: Source image memory.
	`dst_im_mem`: Destination image memory.
Return value	Success: 0; Failure: -1

Example:

```
ret = rknn_lite.im_cvt_color(src, dst)
```

3.15 Device Query and Context Duplication

In a multi-device environment, `get_devices_id()` can be used before initialization to select a device. After Runtime initialization, `rknn3_query()` can query Core count, device memory, I/O, and Tensor attributes. In LLM scenarios, `RKNN3_QUERY_LLM_CONFIG` should also be queried before creating a Session so that Session parameters can be constructed according to the actual model configuration.

get_sdk_version

API	`get_sdk_version`
Function	Gets RKNN SDK version information.
Parameters	None
Return value	SDK version string; returns None on failure.

Example:

```
version = rknn_lite.get_sdk_version()
print(version)
```

get_devices_id

API	`get_devices_id`
Function	Gets the ID list of all connected devices under the specified target.
Parameters	`target`: Specifies the coprocessor type. Default: `rk1820`.
Return value	Device ID list.

Example:

```
devices = rknn_lite.get_devices_id()
print(devices)
```

Note: This interface applies to multi-device deployment. If only one device is used, the first element in the list can be selected directly, or None can be passed where supported by `init_runtime`.

Usage Notes: In a multi-device environment, call this interface before `init_runtime()` and pass the selected device ID through `device_id`. If the business logic does not specify a device, select one available device from the returned list.

dup_context

API	`dup_context`
Function	Duplicates the current RKNN3 Context.
Parameters	None
Return value	`(ret, new_context)`

Example:

```
ret, new_ctx = rknn_lite.dup_context()
```

rknn3_query

API	`rknn3_query`
Function	General query interface used after Runtime initialization to query model, device, performance, dynamic Shape, LLM configuration, LoRA, and post-processing information.
Parameters	`cmd`: `RKNN3QueryCmd` enumeration value.
	`index`: Index value. Used for input/output Tensor index, Core ID, Shape ID, and similar scenarios. Default: 0.
Return value	Query result. Return types for different `cmd` values are as follows:
	`RKNN3_QUERY_IN_OUT_NUM`: `RKNN3InputOutputNum`
	`RKNN3_QUERY_INPUT_ATTR`: `RKNN3TensorAttr`
	`RKNN3_QUERY_OUTPUT_ATTR`: `RKNN3TensorAttr`
	`RKNN3_QUERY_NATIVE_INPUT_ATTR` / `RKNN3_QUERY_NATIVE_OUTPUT_ATTR`: `RKNN3TensorAttr`
	`RKNN3_QUERY_NATIVE_NHWC_INPUT_ATTR` / `RKNN3_QUERY_NATIVE_NHWC_OUTPUT_ATTR`: `RKNN3TensorAttr`
	`RKNN3_QUERY_SDK_VERSION`: `RKNN3SDKVersion`
	`RKNN3_QUERY_CORE_NUMBER`: Core count (`int`)
	`RKNN3_QUERY_CORE_MEM_SIZE`: `RKNN3CoreMemSize`
	`RKNN3_QUERY_DEVICE_MEM_INFO`: `RKNN3DevMemInfo`
	`RKNN3_QUERY_CUSTOM_STRING`: `RKNN3CustomString`
	`RKNN3_QUERY_PERF_DETAIL`: `RKNN3PerfDetail`

API	`rknn3_query`
	`RKNN3_QUERY_PERF_RUN`: `RKNN3PerfRun`
	`RKNN3_QUERY_DYNAMIC_SHAPE_CONFIG`: `RKNN3ShapeConfig`
	`RKNN3_QUERY_DYNAMIC_SHAPE_INFO`: `RKNN3ShapeInfo`
	`RKNN3_QUERY_LLM_CONFIG`: `RKNN3LLMConfig`
	`RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM`: `RKNN3InputOutputNum`
	`RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR`: `RKNN3TensorAttr`
	`RKNN3_QUERY_POSTPROCESS_DYNAMIC_SHAPE_INFO`: `RKNN3ShapeInfo`
	`RKNN3_QUERY_LORA_NUM`: LoRA count (`int`)
	`RKNN3_QUERY_LORA_INFO`: `RKNN3Lora` list
	`RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO`: `RKNN3KvCacheLenGroupInfo` list (allocated by Core count, one element per NPU Core)
	`RKNN3_QUERY_ALLOCATION_INFO`: `RKNN3AllocationInfo` list (allocated by Core count, one element per NPU Core)
	`RKNN3_QUERY_KVCACHE_PAGE_INFO`: `RKNN3KvCachePageInfo` (Core ID specified by `index`)
	`RKNN3_QUERY_ATTENTION_CONFIG`: `RKNN3AttentionConfig` list (count from `n_attention_configs` in `RKNN3_QUERY_LLM_CONFIG`)

Example:

```
version = rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_SDK_VERSION)
io_num = rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_IN_OUT_NUM)
input_attr = rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_INPUT_ATTR, index=0)
```

Usage Notes:

- General models commonly query `IN_OUT_NUM`, `INPUT_ATTR`, `OUTPUT_ATTR`, `DEVICE_MEM_INFO`, and `CORE_NUMBER`.
- LLM applications normally also query `LLM_CONFIG` before Session initialization and use actual values such as `vocab_size`, `embedding_dim`, `max_ctx_len`, `max_position_embeddings`, and `task_type` to construct application parameters.
- Metadata for dynamic Shape, LoRA, and KVCache grouping is also obtained through the corresponding Query Cmd.
- For paged KV Cache, first enumerate Cores using `RKNN3_QUERY_ALLOCATION_INFO`, then call `RKNN3_QUERY_KVCACHE_PAGE_INFO` for each Core (pass its `core_id` through `index`) to obtain page geometry. Use `RKNN3_QUERY_ATTENTION_CONFIG` to obtain settings such as `tokens_per_page`. See `set_kvcache_pages()` in Section 3.10 for the complete procedure.

4. RKNN3 Type Definitions

This chapter describes in detail the type definitions, structures, enumerations, and constants used by RKNN3 Toolkit Lite. These definitions are based on the `rknn3_types.py` file and are used to describe RKNN3 API data structures and parameters.

4.1 Constants, Status Codes, and Enumerations

This section describes public constants, RKNN3 Runtime status codes, and various enumeration definitions used by RKNN3 Toolkit Lite. These definitions are mainly used for error handling, Query, Tensor interpretation, memory synchronization, NPU Core selection, KVCache, and LLM runtime state. A common usage pattern is to query model metadata through `RKNN3QueryCmd`, interpret Tensor attributes using types such as `RKNN3TensorType` and `RKNN3TensorLayout`, and then prepare input and runtime parameters accordingly.

4.1.1 Constant Definitions

RKNN3 Toolkit Lite defines the following constants:

Name	Description
`RKNN3_MAX_DIMS`	`16` : Maximum number of Tensor dimensions
`RKNN3_MAX_STRIDE_DIMS`	`RKNN3_MAX_DIMS + 1` : Maximum number of Tensor stride dimensions
`RKNN3_MAX_NAME_LEN`	`256` : Maximum name length
`RKNN3_MAX_DYNAMIC_SHAPE_NUM`	`512` : Maximum number of dynamic Shapes
`RKNN3_MAX_LORA_NUM`	`32` : Maximum number of LoRA adapters
`RKNN3_MAX_DEVS`	`64` : Maximum number of devices
`RKNN3_MAX_DEV_LEN`	`64` : Maximum device-ID length
`RKNN3_MAX_NPU_NODE_NUM`	`128` : Maximum number of NPU nodes

Name	Description
`RKNN3_MAX_KV_CACHE_LEN_GROUPS`	`16` : Maximum number of length groups supported for one attention type or KV Cache group information
`RKNN3_MAX_ATTENTION_TYPE_NUM`	`3` : Maximum number of attention types supported by LLM configuration
`RKNN3_MAX_CACHE_PAGE_TYPES`	`16` : Maximum number of cache types supported by paged KV Cache
`RKNN3_MAX_MROPE_SECTION_LEN`	`8` : Maximum number of elements in an mRoPE section
`RKNN3_MAX_ATTENTION_CONFIG_NUM`	`32` : Maximum number of attention configurations
`RKNN3_MAX_SPECIAL_BOS_ID_NUM`	`64` : Maximum number of special BOS token IDs
`RKNN3_MAX_SPECIAL_EOS_ID_NUM`	`64` : Maximum number of special EOS token IDs
`LIBRKNN3RT_PATHS`	Default candidate paths for the RKNN3 Runtime libraries. On Linux: `['/usr/lib/librknn3_api.so', '/usr/lib/librknn3_api_rkcp.so', '/usr/lib/librknn3_api_native.so']` ; on macOS, the corresponding `.dylib` files under `/usr/local/lib/` .
`rknn3_runtime_api_ret_code`	Mapping from RKNN3 Runtime return codes to status names. See 4.1.2 Status Codes for complete status codes and meanings.

Runtime libraries are searched in the following priority order:

1. Directories specified by the `LD_LIBRARY_PATH` environment variable;
2. The package directory containing `rknn3lite/api/rknn3_types.py` ;
3. System default paths listed in `LIBRKNN3RT_PATHS` .

If the Runtime library is located in a custom directory, set `LD_LIBRARY_PATH` before starting the Python program:

```
export LD_LIBRARY_PATH=/path/to/rknn3/lib:${LD_LIBRARY_PATH}
```

The directory can contain `librknn3_api.so`, `librknn3_api_rkcp.so`, or `librknn3_api_native.so`. The program searches for available library files in the order above.

4.1.2 Status Codes

RKNN3 Runtime uses integer status codes to represent API execution results. `0` means success; negative values indicate errors or warnings.

High-level RKNN3 Toolkit Lite Python APIs usually log errors according to the underlying Runtime return value and then return `0`, `-1`, `None`, or another result according to the API contract. Therefore, the following table is mainly used to interpret low-level RKNN3 Runtime status codes shown in logs. The actual Python API return form remains subject to the corresponding API description in Chapter 3.

Status Code	Value	Description
`RKNN3_SUCCESS`	0	Operation succeeded
`RKNN3_ERR_FAIL`	-1	Operation failed
`RKNN3_ERR_ARGUMENT_INVALID`	-2	Invalid argument
`RKNN3_ERR_MODEL_INVALID`	-3	Invalid model
`RKNN3_ERR_CTX_INVALID`	-4	Invalid Context
`RKNN3_ERR_RUN_TASK_FAILED`	-5	Task execution failed
`RKNN3_ERR_OUT_OF_MEMORY`	-6	Out of memory
`RKNN3_ERR_TIMEOUT`	-7	Execution timed out
`RKNN3_ERR_INPUT_INVALID`	-8	Invalid input
`RKNN3_ERR_OUTPUT_INVALID`	-9	Invalid output
`RKNN3_ERR_DEVICE_UNAVAILABLE`	-10	Device unavailable
`RKNN3_ERR_DEVICE_UNMATCH`	-11	Device mismatch
`RKNN3_ERR_TARGET_PLATFORM_UNMATCH`	-12	Target platform mismatch

Status Code	Value	Description
`RKNN3_ERR_COMMUNICATION`	-13	Communication error
`RKNN3_ERR_MEM_SYNC_FAILED`	-14	Memory synchronization failed
`RKNN3_ERR_KEY_INVALID`	-15	Invalid decryption key or RSA decryption failed
`RKNN3_ERR_DECRYPT_FAILED`	-16	Model decryption failed, possibly because of a mismatched key or corrupted data
`RKNN3_ERR_WEIGHT_LOAD_NOT_PREPARED`	-17	Streaming weight loading is not prepared; the low-level Runtime must complete model initialization first
`RKNN3_ERR_INCOMPATIBLE_VERSION`	-18	Component versions are incompatible; related Runtime, service, or firmware versions must match
`RKNN3_ERR_MATMUL_INVALID_STATE`	-19	The current MatMul task state does not permit this operation
`RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN`	-20	The completion state of the MatMul task is unknown, and the related memory may still be in use. Keep the associated resources until the task is successfully destroyed before releasing them
`RKNN3_WARN_NPU_CORE_UNUSED`	-100	NPU Core is unused. This is only a warning and does not affect model execution

Usage Notes:

- In general, status code `0` means the underlying Runtime operation succeeded;
- `-1` through `-20` are error states. Locate the cause using the API call site and logs;
- Although `RKNN3_WARN_NPU_CORE_UNUSED` is negative, it is a warning and does not indicate model-execution failure;
- A high-level Python API may convert different low-level errors uniformly into `-1` or `None`. Use the Runtime status code from terminal output or log files for further diagnosis;
- `RKNN3_ERR_INCOMPATIBLE_VERSION` normally indicates a version mismatch among RKNN3 Runtime-related components. Check version compatibility among the Python Wheel, `librknn3_api.so`, device-side services, and coprocessor firmware.

4.1.3 RKNN3QueryCmd

Query command enumeration:

Name	Value	Description
`RKNN3_QUERY_IN_OUT_NUM`	`0`	Query input/output count
`RKNN3_QUERY_INPUT_ATTR`	`1`	Query input attributes
`RKNN3_QUERY_OUTPUT_ATTR`	`2`	Query output attributes
`RKNN3_QUERY_PERF_DETAIL`	`3`	Query detailed performance information
`RKNN3_QUERY_PERF_RUN`	`4`	Query runtime performance
`RKNN3_QUERY_SDK_VERSION`	`5`	Query SDK version
`RKNN3_QUERY_CORE_MEM_SIZE`	`6`	Query Core memory size
`RKNN3_QUERY_CUSTOM_STRING`	`7`	Query custom string
`RKNN3_QUERY_ORIGIN_INPUT_ATTR`	`8`	Query original input attributes
`RKNN3_QUERY_ORIGIN_OUTPUT_ATTR`	`9`	Query original output attributes
`RKNN3_QUERY_NATIVE_INPUT_ATTR`	-	Query native input attributes; retained for backward-compatible naming
`RKNN3_QUERY_NATIVE_OUTPUT_ATTR`	-	Query native output attributes; retained for backward-compatible naming
`RKNN3_QUERY_NATIVE_NC1HWC2_INPUT_ATTR`	-	Query NC1HWC2 input attributes; retained for backward-compatible naming
`RKNN3_QUERY_NATIVE_NC1HWC2_OUTPUT_ATTR`	-	Query NC1HWC2 output attributes; retained for backward-compatible naming
`RKNN3_QUERY_NATIVE_NHWC_INPUT_ATTR`	`10`	Query NHWC input attributes
`RKNN3_QUERY_NATIVE_NHWC_OUTPUT_ATTR`	`11`	Query NHWC output attributes
`RKNN3_QUERY_DEVICE_MEM_INFO`	`12`	Query device memory information
`RKNN3_QUERY_CORE_NUMBER`	`13`	Query Core count
`RKNN3_QUERY_ALLOCATION_INFO`	`14`	Query allocation information
`RKNN3_QUERY_DYNAMIC_SHAPE_CONFIG`	`15`	Query dynamic Shape configuration

Name	Value	Description
`RKNN3_QUERY_DYNAMIC_SHAPE_INFO`	`16`	Query dynamic Shape information
`RKNN3_QUERY_LLM_CONFIG`	`17`	Query LLM configuration
`RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM`	`18`	Query post-processing input/output count
`RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR`	`19`	Query post-processing output attributes
`RKNN3_QUERY_POSTPROCESS_DYNAMIC_SHAPE_INFO`	`20`	Query post-processing dynamic Shape information
`RKNN3_QUERY_LORA_NUM`	`21`	Query LoRA count
`RKNN3_QUERY_LORA_INFO`	`22`	Query LoRA information
`RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO`	`23`	Query KV Cache length-group information
`RKNN3_QUERY_KVCACHE_PAGE_INFO`	`24`	Query paged KV Cache information for a specified Core
`RKNN3_QUERY_ATTENTION_CONFIG`	`25`	Query all attention configurations
`RKNN3_QUERY_CMD_MAX`	`26`	Maximum query-command value

Note: `RKNN3_QUERY_PERF_DETAIL` and `RKNN3_QUERY_PERF_RUN` are no longer present in the current public C header and are retained in the Python layer only for compatibility with older SO versions.

Usage Notes:

- `RKNN3_QUERY_IN_OUT_NUM` is normally the first query after initialization and determines how many input/output attributes need to be read subsequently.
- `RKNN3_QUERY_INPUT_ATTR` / `OUTPUT_ATTR` are used together with `index`.
- `RKNN3_QUERY_LLM_CONFIG` can obtain configuration such as `vocab_size`, context length, and model type, avoiding application-side hard-coding.
- `RKNN3_QUERY_DEVICE_MEM_INFO` and `CORE_NUMBER` can be used for device-resource checks and for generating an appropriate `core_mask`.
- `RKNN3_QUERY_KVCACHE_PAGE_INFO` and `RKNN3_QUERY_ATTENTION_CONFIG` are used for paged KV Cache. First enumerate the Cores using `RKNN3_QUERY_ALLOCATION_INFO`; see `set_kvcache_pages()` in Section 3.10.

4.1.4 RKNN3TensorType

Tensor data-type enumeration:

Name	Value	Description
`RKNN3_TENSOR_FLOAT32`	`0`	32-bit floating point
`RKNN3_TENSOR_FLOAT16`	`1`	16-bit floating point
`RKNN3_TENSOR_INT8`	`2`	8-bit signed integer
`RKNN3_TENSOR_UINT8`	`3`	8-bit unsigned integer
`RKNN3_TENSOR_INT16`	`4`	16-bit signed integer
`RKNN3_TENSOR_UINT16`	`5`	16-bit unsigned integer
`RKNN3_TENSOR_INT32`	`6`	32-bit signed integer
`RKNN3_TENSOR_UINT32`	`7`	32-bit unsigned integer
`RKNN3_TENSOR_INT64`	`8`	64-bit signed integer
`RKNN3_TENSOR_UINT64`	`9`	64-bit unsigned integer
`RKNN3_TENSOR_BOOL`	`10`	Boolean type
`RKNN3_TENSOR_INT4`	`11`	4-bit signed integer
`RKNN3_TENSOR_FLOAT8E4M3FN`	`12`	8-bit floating-point format E4M3FN
`RKNN3_TENSOR_BFLOAT16`	`13`	BF16 floating-point format
`RKNN3_TENSOR_FLOAT8E8M0`	`14`	8-bit floating-point format E8M0
`RKNN3_TENSOR_FLOAT4E2M1`	`15`	4-bit floating-point format E2M1
`RKNN3_TENSOR_TYPE_MAX`	`16`	Maximum Tensor-type value

Usage Notes: Applications can select the NumPy input type according to `RKNN3TensorAttr.dtype`. Common mappings include FP16→`numpy.float16`, FP32→`numpy.float32`, INT8→`numpy.int8`, UINT8→`numpy.uint8`, and INT32→`numpy.int32`. For runtime formats not directly supported by NumPy, high-level interfaces normally use a convertible logical data type.

4.1.5 RKNN3TensorQntType

Tensor quantization-type enumeration:

Name	Value	Description
`RKNN3_TENSOR_QNT_NONE`	`0`	No quantization
`RKNN3_TENSOR_PER_LAYER_SYMMETRIC`	`1`	Per-layer symmetric quantization
`RKNN3_TENSOR_PER_LAYER_ASYMMETRIC`	`2`	Per-layer asymmetric quantization
`RKNN3_TENSOR_PER_CHANNEL_SYMMETRIC`	`3`	Per-channel symmetric quantization
`RKNN3_TENSOR_PER_CHANNEL_ASYMMETRIC`	`4`	Per-channel asymmetric quantization
`RKNN3_TENSOR_PER_GROUP_SYMMETRIC`	`5`	Per-group symmetric quantization
`RKNN3_TENSOR_PER_GROUP_ASYMMETRIC`	`6`	Per-group asymmetric quantization

4.1.6 RKNN3TensorLayout

Tensor-layout enumeration:

Name	Value	Description
`RKNN3_TENSOR_UNDEFINED`	`0`	Undefined
`RKNN3_TENSOR_NCHW`	`1`	NCHW format
`RKNN3_TENSOR_NHWC`	`2`	NHWC format
`RKNN3_TENSOR_NC1HWC2`	`3`	NC1HWC2 format
`RKNN3_TENSOR_CHWN`	`4`	CHWN format
`RKNN3_TENSOR_HWIO`	`5`	HWIO format
`RKNN3_TENSOR_OIHW`	`6`	OIHW format
`RKNN3_TENSOR_O1I1HWI2O2`	`7`	O1I1HWI2O2 format

Usage Notes:

- `NCHW` and `NHWC` are the most common logical layouts for high-level `inference(data_format=...)`.
- `NC1HWC2` is an internal/native packed Runtime layout. Applications generally do not construct this packed buffer directly and instead provide logical NCHW input as required by the high-level interface.
- When matching 4-D dynamic-Shape input, consider both NCHW and NHWC logical representations to avoid incorrectly treating the same logical Shape as mismatched only

because of dimension ordering.

4.1.7 RKNN3MemAllocFlags

Memory-allocation flag enumeration:

Name	Value	Description
`RKNN3_FLAG_MEMORY_FLAGS_DEFAULT`	`0`	Default memory flags
`RKNN3_FLAG_MEMORY_CACHEABLE`	`1`	Cacheable memory
`RKNN3_FLAG_MEMORY_NON_CACHEABLE`	`2`	Non-cacheable memory

4.1.8 RKNN3MemSyncMode

Memory-synchronization mode enumeration:

Name	Value	Description
`RKNN3_MEMORY_SYNC_TO_DEVICE`	`1`	Synchronize to device
`RKNN3_MEMORY_SYNC_FROM_DEVICE`	`2`	Synchronize from device
`RKNN3_MEMORY_SYNC_BIDIRECTIONAL`	`3`	Bidirectional synchronization

4.1.9 RKNN3CoreMask

NPU Core-mask enumeration:

Name	Value	Description
`RKNN3_NPU_CORE_AUTO`	`0`	Automatically select Cores
`RKNN3_NPU_CORE_0`	`1`	Core 0
`RKNN3_NPU_CORE_1`	`2`	Core 1
`RKNN3_NPU_CORE_2`	`4`	Core 2
`RKNN3_NPU_CORE_3`	`8`	Core 3
`RKNN3_NPU_CORE_4`	`16`	Core 4
`RKNN3_NPU_CORE_5`	`32`	Core 5
`RKNN3_NPU_CORE_6`	`64`	Core 6
`RKNN3_NPU_CORE_7`	`128`	Core 7
`RKNN3_NPU_CORE_ALL`	`0xFFFFFFFF`	Automatically use the NPU Cores supported by the platform

Usage Notes: Core Mask is a bit mask in which each bit represents the corresponding NPU Core. The application can first obtain `RKNN3_QUERY_CORE_NUMBER`, then generate a mask according to the available Cores and verify that a user-provided mask matches the actual Core configuration.

4.1.10 RKNN3KVCachePolicy

KVCache-policy enumeration:

Name	Value	Description
`RKNN3_KVCACHE_POLICY_DEFAULT`	`0`	Default policy (equivalent to RECURRENT)
`RKNN3_KVCACHE_POLICY_RECURRENT`	`1`	Recurrent policy
`RKNN3_KVCACHE_POLICY_NORMAL`	`2`	Normal policy; uses only KV Cache with `max_context_len` length
`RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT`	`0x10` `0`	Checkpoint-save policy; only effective for models containing linear attention or sliding attention, such as the Qwen3.5 and Gemma4 series

Usage Notes:

- `NORMAL`: Standard context-cache mode.
- `RECURRENT`: Mode that cyclically reuses cache space.
- `DEFAULT`: Currently equivalent to the recurrent default policy.
- `SAVE_CHECKPOINT`: Must be used together with `RKNN3KVCachePolicyParam.save_checkpoint`.

4.1.11 RKNN3KVCacheClearPolicy

KVCache-clear policy enumeration:

Name	Value	Description
`RKNN3_KVCACHE_CLEAR_ALL`	`0`	Clear all
`RKNN3_KVCACHE_KEEP_SYSTEM_PROMPT`	`1`	Keep system prompt
`RKNN3_KVCACHE_KEEP_N_TOKENS`	`2`	Clear KVCache while retaining the specified number of tokens

Usage Notes: `CLEAR_ALL` is suitable for independent requests or for a full reset when context is nearly exhausted. `KEEP_SYSTEM_PROMPT` is suitable for repeated tests/conversations that reuse a fixed system-prompt cache and reduce repeated Prefill. `KEEP_N_TOKENS` requires the number of

retained tokens to be specified through `RKNN3KVCacheClearPolicyParam.keep_n_tokens.n_tokens`.

4.1.12 RKNN3LLMInputType

LLM input-type enumeration:

Name	Value	Description
`RKNN3_LLM_INPUT_PROMPT`	`0`	Prompt input
`RKNN3_LLM_INPUT_TOKEN`	`1`	Token input
`RKNN3_LLM_INPUT_EMBED`	`2`	Embedding input
`RKNN3_LLM_INPUT_MULTIMODAL`	`3`	Multimodal input
`RKNN3_LLM_INPUT_AUX`	`4`	Auxiliary input

Usage Notes: This enumeration corresponds to the main input paths of `session_run()`: text Prompt, Token ID, precomputed Embed, multimodal input, and AUX input. The high-level interface constructs the corresponding low-level input type according to the actual arguments passed.

4.1.13 LLMCallState

LLM callback-state enumeration:

Name	Value	Description
`RKLLM_RUN_NORMAL`	`0`	Running normally
`RKLLM_RUN_WAITING`	`1`	Waiting
`RKLLM_RUN_FINISH`	`2`	Run finished
`RKLLM_RUN_STOP`	`3`	Run stopped
`RKLLM_RUN_MAX_NEW_TOKEN_REACHED`	`4`	Maximum new-token count reached
`RKLLM_RUN_ERROR`	`5`	Runtime error
`RKLLM_RUN_PAUSE`	`6`	Run paused
`RKLLM_RUN_RESUME`	`7`	Run resumed

Usage Notes: A result Callback should at minimum handle NORMAL, FINISH, STOP, MAX_NEW_TOKEN_REACHED, and ERROR. When pause/resume is used, PAUSE and RESUME should also be handled. During streaming decoding, WAITING means the current token fragment is not yet sufficient to form a complete UTF-8 character; wait for subsequent tokens before outputting text.

4.1.14 RKNN3GrammarType

Grammar-type enumeration:

Name	Value	`grammar_str` Content	Description
`RKNN3_GRAMMAR_JSON_SCHEMA`	`0`	JSON Schema text	Constrains output to JSON that conforms to the Schema. Suitable for information extraction, fixed API responses, and classification results.
`RKNN3_GRAMMAR_EBNF`	`1`	EBNF rule text	Uses EBNF to define custom grammar, such as a fixed set of enumeration labels.
`RKNN3_GRAMMAR_BUILTIN_JSON`	`2`	Normally `None`	Uses the built-in JSON Grammar to constrain output to any valid JSON.
`RKNN3_GRAMMAR_REGEX`	`3`	Regular-expression text	Constrains output to strings matching a regular expression, such as dates, order numbers, or email addresses.
`RKNN3_GRAMMAR_STRUCTURAL_TAG`	`4`	Structural Tag JSON description text	Constrains output to a structure composed of constant text, JSON Schema, and similar elements. Suitable for Tool / Function Calling scenarios.

Usage Notes: Pass the enumeration value to the `grammar_type` parameter of `set_grammar()`. The `grammar_str` formats for different types must not be mixed. Builtin JSON does not require rule text.

4.1.15 Other Enumeration Types

In addition to the enumerations above, the current version also defines the following enumeration types:

Name	Description
`RKNN3MemType`	`RKNN3_MEMORY_TYPE_NPU_DRAM = 0 << 0`, `RKNN3_MEMORY_TYPE_EXT_DDR = 1 << 0`
`RKNN3KVCacheDtype`	KV Cache data type, including `UNDEFINED`, `INT4_TO_F16`, `INT4_TO_F8`, `INT8_TO_F16`, `FLOAT4_TO_F16`, `FLOAT4_TO_F8`, `FLOAT8_TO_F16`, `FLOAT8_TO_F8`, `FLOAT16`
`RKNN3KVCacheStoreMethod`	KV Cache storage method, including `UNDEFINED`, `NORMAL`, `GROUP_QUANT`
`RKNN3AttentionType`	Attention type, including `RKNN3_ATTENTION_TYPE_FULL_ATTENTION = 0`, `RKNN3_ATTENTION_TYPE_SLIDING_ATTENTION = 1`, `RKNN3_ATTENTION_TYPE_LINEAR_ATTENTION = 2`

Name	Description
`RKNN3PageCacheType`	Paged KV Cache type. Values match the model schema: `RKNN3_PAGE_CACHE_GENERAL = 11`, `RKNN3_PAGE_CACHE_FULL_ATTENTION = 12`, `RKNN3_PAGE_CACHE_SLIDING_ATTENTION = 13`, `RKNN3_PAGE_CACHE_LINEAR_ATTENTION = 14`, `RKNN3_PAGE_CACHE_CONV_STREAMING = 15`, `RKNN3_PAGE_CACHE_FULL_RESIDUAL = 17`, `RKNN3_PAGE_CACHE_SLIDING_RESIDUAL = 18`
`LLMOutputCallbackState`	Output-callback stages, including `PREFILL_PROCESSING`, `PREFILL_FINISHED`, `DECODE_PROCESSING`, `DECODE_FINISHED`
`RKNN3LLMTaskType`	LLM task type, including `RKNN3_LLM_TASK_GENERATE = 0`, `RKNN3_LLM_TASK_EMBEDDING = 1`, `RKNN3_LLM_TASK_RERANKER = 2`
`RKNN3ImFmt`	Image formats, including `RGB888`, `BGR888`, `GRAY8`, `YCbCr/YCrCb 420/422 SP`, `UNKNOWN`
`RKNN3ImProcFlag`	Image-processing flags, including `CROP`, `RESIZE`, `FILL`, `COLOR_SPACE_CONVERT`, `DECODE`, `ENCODE`
`RKNN3OpTargetType`	Custom-operator target type; currently includes `RKNN3_OP_TARGET_TYPE_CPU = 1`
`RKNN3OpPluginType`	Custom-operator plugin type, including `POSTPROCESS` and `CUSTOM_OP`

`LLMOutputCallbackState` Usage Notes:

- `PREFILL_PROCESSING`: Prefill is in progress;
- `PREFILL_FINISHED`: Prefill is complete; suitable for reading outputs that are valid only after Prefill finishes;
- `DECODE_PROCESSING`: Decode is in progress;
- `DECODE_FINISHED`: Decode is complete.

The application should check `state` for the required stage before reading output Tensors provided by `LLMOutputCallback`, rather than assuming that the data has the same meaning in every Callback.

4.2 General Structures

These structures are mostly returned by `rknn3_query()` and describe model I/O count, dynamic Shape, SDK/device information, and Runtime configuration. Applications normally read their fields rather than directly modifying Runtime-internal data.

4.2.1 RKNN3InputOutputNum

Input/output-count information:

Field	Type	Description
`n_input`	`uint32`	Number of inputs
`n_output`	`uint32`	Number of outputs

Usage Notes: Returned by `RKNN3_QUERY_IN_OUT_NUM`. Applications normally read `n_input` / `n_output` first and then query each `RKNN3TensorAttr` by index.

4.2.2 RKNN3PerfDetail

Performance details:

Field	Type	Description
`perf_data`	`char*`	Performance data
`data_len`	`uint64`	Data length

4.2.3 RKNN3PerfRun

Runtime performance:

Field	Type	Description
`run_duration`	`int64`	Run duration

4.2.4 RKNN3SDKVersion

SDK version:

Field	Type	Description
`api_version`	`char[256]`	API version
`drv_version`	`char[256]`	Driver version

4.2.5 RKNN3CustomString

Custom string:

Field	Type	Description
`string`	`char[1024]`	String

4.2.6 RKNN3Config

RKNN3 configuration:

Field	Type	Description
`priority`	`int32`	Priority
`run_timeout`	`uint32`	Runtime timeout
`run_core_mask`	`uint32`	Runtime Core mask
`user_mem_weight`	`uint8`	User weight memory
`user_mem_internal`	`uint8`	User internal memory
`user_sram`	`uint8`	User SRAM

4.2.7 RKNN3ShapeInfo

Shape information:

Field	Type	Description
`shape_id`	`int32`	Shape ID
`n_inputs`	`uint32`	Number of inputs
`input_attrs`	`RKNN3TensorAttr*`	Input attributes
`n_outputs`	`uint32`	Number of outputs
`output_attrs`	`RKNN3TensorAttr*`	Output attributes
`is_default`	`uint8`	Whether this is the default Shape

Usage Notes: Describes one dynamic Shape combination and its input/output attributes. Applications can read `shape_id` and input Shape and match them against the actual input Shape.

4.2.8 RKNN3ShapeConfig

Shape configuration:

Field	Type	Description
`n_shapes`	`uint32`	Number of Shapes
`current_shape_id`	`int32`	Current Shape ID

Usage Notes: Describes how many dynamic Shape groups the model has and which Shape is currently active. High-level `get_dynamic_shape_infos()` organizes the low-level dynamic Shape query result into a structure that is easier to use from Python.

4.2.9 RKNN3InitExtend

Extended initialization parameters:

Field	Type	Description
`device_id`	`char*`	Device ID
`reserved`	`uint8[128]`	Reserved

4.2.10 RKNN3NodeMemInfo

Node memory information:

Field	Type	Description
`total`	`uint64`	Total memory
`free`	`uint64`	Free memory

4.2.11 RKNN3DevMemInfo

Device memory information:

Field	Type	Description
`node_num`	`uint32`	Number of nodes
`sys_total`	`uint64`	Total system memory
`sys_free`	`uint64`	Free system memory
`node_mem_info`	`RKNN3NodeMemInfo[128]`	Node memory information

Usage Notes: `RKNN3_QUERY_DEVICE_MEM_INFO` can be used to read `sys_total/sys_free` and memory information for each node, which is useful for checking available device memory before execution or diagnosing OOM.

4.2.12 RKNN3Device

Device information:

Field	Type	Description
`id`	`char[64]`	Device ID
`type`	`char[64]`	Device type
`mem_info`	`RKNN3DevMemInfo`	Memory information

4.2.13 RKNN3Devices

Device list:

Field	Type	Description
`n_devices`	`uint32`	Number of devices
`devices`	`RKNN3Device[64]`	Device information

4.2.14 Custom-Operator Structures

Name	Description
`RKNN3CustomOpContext`	Custom-operator context. Fields include `rknn_ctx`, `priv_data`, and `user_data`.
`RKNN3CustomOp`	Custom-operator registration information. Fields include `op_type`, `plugin_type`, `target`, `version`, `author`, `description`, and callbacks such as `init/prepare/compute/deinit/get_output_num/get_attrs`.

4.3 Tensor and Memory Structures

Tensor structures connect "Tensor attributes" and "actual memory". Ordinary high-level inference only requires `numpy.ndarray`; explicit-memory and Callback scenarios use `RKNN3TensorAttr`, `RKNN3TensorMemory`, and `RKNN3Tensor` together.

4.3.1 RKNN3QuantInfo

Quantization information:

Field	Type	Description
`scale`	`float`	Scale factor
`zero_point`	`int32`	Zero point

4.3.2 RKNN3TensorAttr

Tensor attribute information:

Field	Type	Description
`index`	`uint32`	Index
`name`	`char[256]`	Name
`n_dims`	`uint32`	Number of dimensions
`shape`	`uint32[16]`	Shape
`aligned_size`	`uint64`	Aligned size
`n_stride`	`uint32`	Number of stride dimensions
`stride`	`uint64[17]`	Stride
`n_elems`	`uint32`	Number of elements
`dtype`	`int`	Data type
`layout`	`int`	Layout
`qnt_type`	`int`	Quantization type
`qnt_info`	`RKNN3QuantInfo`	Quantization information
`n_orig_dims`	`uint32`	Number of original Tensor dimensions
`orig_shape`	`uint32[16]`	Original Tensor Shape
`orig_layout`	`int`	Original Tensor Layout
`core_id`	`int32`	Core ID

Usage Notes:

- Only the first `n_dims` entries of `shape` are valid. Similarly, read `stride` together with `n_stride`/actual dimensions;

- `aligned_size` is the buffer size calculated by Runtime according to alignment requirements. When creating explicit memory, prefer this value instead of simply multiplying the element count;
- `dtype`, `layout`, and `qnt_type/qnt_info` determine how data is interpreted. Input preparation, dynamic Shape matching, and performance analysis should all use these fields as the reference;
- `core_id` identifies the NPU Core associated with a Tensor and is used to select the target Core in multi-Core user-memory management;
- For Output Callback, normally query this structure through `RKNN3_QUERY_OUTPUT_ATTR` first and then call `create_mem()` to prepare a buffer for the corresponding output Tensor.

4.3.3 RKNN3MemSize

Memory-size information:

Field	Type	Description
`total_const_size`	`uint64`	Total constant size
`total_internal_size`	`uint64`	Total internal-memory size
`total_dma_allocated_size`	`uint64`	Total DMA-allocated size
`total_sram_size`	`uint64`	Total SRAM size
`free_sram_size`	`uint64`	Free SRAM size

4.3.4 RKNN3TensorMemory

Tensor memory:

Field	Type	Description
`virt_addr`	`void*`	Virtual address
`phys_addr`	`uint64`	Physical address
`fd`	`int32`	File descriptor
`buffer_id`	`uint64`	Buffer ID
`offset`	`uint64`	Offset
`size`	`uint64`	Size
`flags`	`uint64`	Flags
`core_id`	`int32`	Core ID
`priv_data`	`void*`	Private data
`mem_type`	`int`	Memory type

Usage Notes:

- `virt_addr` is used for direct CPU-side access to the Tensor buffer, while `size` indicates the actual allocated byte count;
- `phys_addr`, `fd`, and `offset` are used in external-memory scenarios such as physical memory and DMA-BUF/shared memory;
- In Output Callback, this structure is normally obtained through `RKNN3Tensor.mem`, and `virt_addr` is converted to the corresponding data type according to `attr.dtype` for reading;
- Memory created by `create_mem()` should be released through `destroy_mem()` when no longer used. Do not directly free `virt_addr`.

4.3.5 RKNN3Tensor

RKNN3 Tensor:

Field	Type	Description
`mem`	`RKNN3TensorMemory*`	Memory
`attr`	`RKNN3TensorAttr*`	Attributes

Usage Notes: `RKNN3Tensor` does not directly store Tensor data. Instead, it associates metadata and buffer through two pointers:

- `attr` points to `RKNN3TensorAttr`, used to interpret output name, Shape, dtype, Layout, etc.;
- `mem` points to `RKNN3TensorMemory`, used to access actual data;
- `output_tensors` received by `LLMOutputCallback` is an array of this structure, so callbacks usually read both `tensor.attr.contents` and `tensor.mem.contents`.

4.3.6 RKNN3AllocationInfo

Memory-allocation information:

Field	Type	Description
`core_id`	`int32`	Core ID
`command_mem`	`RKNN3TensorMemory`	Command memory
`weight_mem`	`RKNN3TensorMemory`	Weight memory
`internal_mem`	`RKNN3TensorMemory`	Internal memory
`kvcache_mem`	`RKNN3TensorMemory`	KVCache memory
`reserved`	`uint8[128]`	Reserved

4.3.7 RKNN3CoreMemSize

Name	Description
`RKNN3CoreMemSize`	Core memory-size information. Fields include `core_id`, `weight_size`, `internal_size`, and `reserved`.

4.4 LLM and VLM Structures

These structures cover LLM parameters, generation results, runtime state, and multimodal input. LLM scenarios commonly use `RKNN3LLMConfig`, `RKLLMResult`, and `RKLLMRunState` directly. In VLM scenarios, a `Float16` pointer can be used to pass the vision-model output embedding into a multimodal input structure.

4.4.1 Float16

16-bit floating-point representation:

Field	Type	Description
`frac`	`uint16, 10 bits`	Fraction
`exp`	`uint16, 5 bits`	Exponent
`sign`	`uint16, 1 bit`	Sign

Usage Notes: In VLM scenarios, a NumPy `float16` output can be converted to `Float16*` using `outputs.ctypes.data_as(ctypes.POINTER(Float16))` and then assigned to `RKNN3Image.image_embed`. Therefore, the corresponding NumPy array must remain alive while the low-level pointer is valid.

4.4.2 RKNN3VocabInfo

Vocabulary information:

Field	Type	Description
`vocab_size`	`int`	Vocabulary size
`special_bos_id`	`int[64]`	Special BOS token IDs
`special_eos_id`	`int[64]`	Special EOS token IDs
`n_special_bos_id`	`int`	Number of special BOS token IDs
`n_special_eos_id`	`int`	Number of special EOS token IDs
`linefeed_id`	`int`	Line-feed token ID
`skip_special_token`	`bool`	Whether to skip special tokens
`ignore_eos_token`	`bool`	Whether to ignore EOS tokens
`reserved`	`uint8[64]`	Reserved

Usage Notes: Fields such as `special_eos_id`, `skip_special_token`, and `ignore_eos_token` directly affect generation termination and output processing. In performance-testing scenarios, `ignore_eos_token` can be enabled to force generation until `max_new_tokens`, making Decode performance measurements more stable.

4.4.3 RKNN3SamplingParams

Sampling parameters:

Field	Type	Description
`top_k`	`int32`	top-k
`top_p`	`float`	top-p
`temperature`	`float`	Temperature
`repeat_penalty`	`float`	Repetition penalty
`frequency_penalty`	`float`	Frequency penalty
`presence_penalty`	`float`	Presence penalty

Usage Notes: `top_k/top_p/temperature` control sampling range and randomness, while `repeat_penalty/frequency_penalty/presence_penalty` control repetition tendency. For more deterministic output, reduce sampling randomness, for example by setting a smaller `top_k`. Performance tests and evaluation scenarios also commonly use more deterministic sampling configurations.

4.4.4 RKNN3LLMTensor

LLM Tensor input:

Field	Type	Description
`name`	`char*`	Name
`prompt`	`char*`	Prompt
`embed`	`Float16*`	Embedding
`tokens`	`int32*`	Tokens
`n_tokens`	`uint64`	Number of tokens
`enable_thinking`	`bool`	Enable thinking

Usage Notes: This structure carries low-level data for pure text/Token/Embedding paths. High-level `session_run(prompt=...)`, `session_run(tokens=...)`, or `session_run(embeds=...)` eventually constructs corresponding input. The three main-input forms must not be set simultaneously.

4.4.5 RKNN3AuxTensor

Auxiliary Tensor (same as `RKNN3Tensor`).

Usage Notes: `RKNN3AuxTensor` is equivalent to a normal `RKNN3Tensor` and represents an additional model input Tensor. At the Python high level, `RKNN3AuxTensorWrapper` is more commonly used to pass NumPy data, with the wrapper constructing the low-level AUX Tensor.

4.4.6 RKNN3LLMMultiModelTensor

Multimodal Tensor input:

Field	Type	Description
`name`	`char*`	Name
`prompt`	`char*`	Prompt
`tokens`	`int32*`	Tokens
`n_tokens`	`uint64`	Number of tokens
`enable_thinking`	`bool`	Enable thinking
`image`	`RKNN3Image`	Image data
`audio`	`RKNN3Audio`	Audio data
`video`	`RKNN3Video`	Video data

4.4.7 RKNN3LLMInputUnion

LLM input union:

Field	Type	Description
`llm_input`	`RKNN3LLMTensor`	LLM input
`multimodal_input`	`RKNN3LLMMultiModelTensor`	Multimodal input
`aux_input`	`RKNN3AuxTensor`	Auxiliary input

4.4.8 RKNN3LLMInput

LLM input:

Field	Type	Description
`role`	`char*`	Role
`input_type`	`int`	Input type
`input_union`	`RKNN3LLMInputUnion`	Input union

4.4.9 RKNN3LLMExtendParam

Extended LLM parameters:

Field	Type	Description
`reserved`	`uint8[128]`	Reserved

4.4.10 RKNN3LLMParam

LLM parameters:

Field	Type	Description
`logits_name`	`char*`	Logits name
`max_context_len`	`int32`	Maximum context length
`sampling_param`	`RKNN3SamplingParams`	Sampling parameters
`vocab_info`	`RKNN3VocabInfo`	Vocabulary information
`extend_param`	`RKNN3LLMExtendParam`	Extended parameters

Usage Notes: During Session initialization, sampling parameters, vocabulary information, and context length from Python dictionaries are converted into this structure. `max_context_len` should use `RKNN3LLMConfig.max_ctx_len` as the reference. If a configured value exceeds the model-supported range, the application should validate it and stop initialization.

4.4.11 RKNN3LLMInferParam

LLM inference parameters:

Field	Type	Description
`keep_history`	`int, 1 means keep, 0 means do not keep`	Whether to retain history context
`max_new_tokens`	`int32`	Maximum number of newly generated tokens
`prefill_only`	`bool`	Whether to perform Prefill only
`disable_sampling`	`bool`	Whether to disable sampling after a single Prefill
`keep_grammar`	`bool`	Whether to keep the grammar matcher state across runs. Default: false. When enabled, the per-request grammar matcher is NOT re-initialized at the start/end of each `rknn3_session_run()`, so the grammar state is retained for continuous constrained generation across multiple runs
`reserved`	`uint8[127]`	Reserved

Usage Notes: The `keep_history`, `max_new_tokens`, `prefill_only`, `disable_sampling`, and `keep_grammar` parameters of `session_run()` can override the Session default inference parameters. Evaluation tasks usually use `keep_history=False`; continuous conversations usually use True. At the high-level Python API, these Boolean parameters also accept integer values such as `0` or `1`; the wrapper automatically converts them to `bool` before setting the corresponding fields in this structure.

4.4.12 RKLLMResult

LLM generation result:

Field	Type	Description
`token_ids`	`int*`	Token IDs
`num_tokens`	`int`	Number of tokens

Usage Notes: In `LLMResultCallback`, use `result_ptr.contents.num_tokens` to obtain the number of tokens in the current callback, and read token IDs from `token_ids`. For streaming output, tokens can be accumulated and decoded together by the Tokenizer to avoid incomplete UTF-8 characters caused by individual tokens.

4.4.13 RKLLMRunState

LLM runtime state:

Name	Description
`n_total_tokens`	Total number of tokens processed so far (`uint64`)
`n_reuse_tokens`	Number of tokens in the current input reused from KV Cache (`uint64`)
`n_max_tokens`	Maximum number of processable tokens (`uint64`)
`n_prefill_tokens`	Number of tokens processed during Prefill (`uint64`)
`n_decode_tokens`	Number of tokens generated during Decode (`uint64`)
`n_input_tokens`	Number of input tokens (`uint64`, same as `n_prefill_tokens`)
`n_output_tokens`	Number of output tokens (`uint64`, normally `n_decode_tokens + 1`)
`input_tokens`	Pointer to input-token array (`int32*`), managed by Runtime and must not be freed by the user
`output_tokens`	Pointer to output-token array (`int32*`), managed by Runtime and must not be freed by the user
`kvcache_policy`	KVCache policy (`int`)
`n_loras_enabled`	Number of enabled LoRA adapters (`int32`)
`loras_enabled`	Array of enabled LoRA adapters (`RKNN3Lora*`)

Usage Notes:

- `n_total_tokens/n_max_tokens` can be used to determine remaining context capacity.
- `n_prefill_tokens/n_decode_tokens` can be used for performance statistics or to validate the current execution phase.
- `n_reuse_tokens` reflects the number of tokens in the current input reused from KVCache.
- `kvcache_policy` and `n_loras_enabled/loras_enabled` can be used to confirm the current Session's cache and LoRA configuration.
- `input_tokens/output_tokens` are managed by Runtime and are read-only to the user; do not free them.

4.4.14 RKLLMResultLastHiddenLayer

Last hidden-layer result:

Field	Type	Description
`hidden_states`	`float*`	Hidden states
`embd_size`	`int`	Embedding size
`num_tokens`	`int`	Number of tokens

4.4.15 RKNN3LLMConfig

Name	Description
`RKNN3LLMConfig`	LLM model configuration. Fields include `chat_template`, `vocab_size`, `embedding_dim`, `max_ctx_len`, `max_position_embeddings`, `kvcache_store_method`, `kvcache_dtype`, `kvcache_group_size`, `kvcache_residual_depth`, `model_type`, `task_type`, `rope_cache_host_storage`, `n_attention_kvcache_lens`, `attention_kvcache_lens`, `n_attention_configs`, and `reserved`.

Usage Notes: This is one of the most important read-only model configurations for LLM applications. It is recommended to query this structure before Session initialization and construct runtime parameters based on its fields:

- `vocab_size`: Validate the tokenizer vocabulary and reshape the embedding table;
- `embedding_dim`: Validate or derive the token-embedding dimension;
- `max_ctx_len`: Use as the basis for the Session `max_context_len`; application settings should not exceed the model-supported range;
- `max_position_embeddings`: Maximum positional-encoding range supported by the model, useful for checking long-context configuration;
- `model_type`, `task_type`: Identify model type and tasks such as Generate/Embedding/Reranker;
- `kvcache_store_method`, `kvcache_dtype`, `kvcache_group_size`, `kvcache_residual_depth`: Describe model KVCache storage and quantization configuration;
- `rope_cache_host_storage`: Indicates whether the model requires application-side Host RoPE Cache management. If non-zero, some models may have RoPE extended inputs that need to be filled by `LLMInputCallback` according to the current position;
- `n_attention_kvcache_lens`, `attention_kvcache_lens`: Describe KVCache length configuration for different attention types;
- `n_attention_configs`: Number of attention-configuration entries in the model. For a paged KV Cache model, this determines the array length to query using `RKNN3_QUERY_ATTENTION_CONFIG`.

Note: The specific names, data formats, and Host Cache file organization of extended inputs are model-dependent and are not fixed conventions of the RKNN3Lite general API.

4.4.16 RKNN3GrammarConfig

Grammar-engine vocabulary configuration:

Field	Type	Description
`vocab_size`	`int32`	Vocabulary size, i.e. the number of token pieces.
`encoded_vocab`	`uint8*`	Vocabulary data encoded in token-ID order. Each token consists of an `int32` length followed by the corresponding token bytes.
`encoded_vocab_len`	`int32`	Total byte length of `encoded_vocab`.
`n_stop_tokens`	`int32`	Number of stop tokens.
`stop_tokens`	`int32*`	Array of stop-token IDs.

Normally, this structure does not need to be filled manually. It is recommended to pass `token_pieces` arranged in token-ID order to `init_grammar()`, and let the Python API construct it automatically:

```
from rknn3lite.api.rknn3_types import RKNN3GrammarConfig

config = RKNN3GrammarConfig.from_token_pieces(
    token_pieces,
    stop_tokens=[tokenizer.eos_token_id],
)
ret = rknn_lite.init_grammar(config=config)
```

`from_token_pieces()` supports `str` and bytes-like token pieces; `str` values are encoded as UTF-8. When constructing the structure manually, ensure that the buffers referenced by `encoded_vocab` and `stop_tokens` remain valid for the entire period during which the current Context uses Grammar. A structure created with `from_token_pieces()` automatically keeps references to its internal buffers.

4.4.17 RKNN3GrammarSetOptions

Grammar setting options:

Field	Type	Description
`any_whitespace`	`int`	Whitespace-handling option.
`indent`	`int`	Formatting-indentation option.
`strict_mode`	`int`	Strict-mode option.
`max_whitespace_cnt`	`int`	Maximum number of consecutive whitespace characters.
`any_order`	`int`	JSON object key-order handling option.
`item_separator`	`char*`	Item separator.
`key_value_separator`	`char*`	Key-value separator.
`root_rule_name`	`char*`	EBNF root-rule name.

Use `RKNN3GrammarSetOptions.default()` to create an options object initialized with Runtime defaults and override fields as needed:

```
from rknn3lite.api.rknn3_types import (
    RKNN3GrammarSetOptions,
    RKNN3GrammarType,
)

options = RKNN3GrammarSetOptions.default(
    root_rule_name='root',
)
ret = rknn_lite.set_grammar(
    'root ::= "yes" | "no"\\n',
    RKNN3GrammarType.RKNN3_GRAMMAR_EBNF,
    options=options,
)
```

String fields support `str`, `bytes`, or `None`; `str` values are automatically encoded as UTF-8. If no Grammar-engine formatting options need to be adjusted, it is recommended to keep `options=None` in `set_grammar()`.

4.5 KVCache Structures

KVCache structures describe cache-policy parameters and cache configuration for different attention types/length groups, including paged-cache geometry and bindings. Ordinary applications normally only select a policy. Additional structures are needed for recurrent, checkpoint, or multi-length-group scenarios.

4.5.1 RECURRENT

Recurrent KVCache parameters:

Field	Type	Description
`n_keep`	`int64`	Number of KV Cache entries to retain under the recurrent policy
`n_keep_aligned`	`int64`	Retained count aligned according to `kvcache_group_size`

Note: If the model contains a system prompt, Runtime automatically uses the system-prompt length and `n_keep` and `n_keep_aligned` do not take effect.

Usage Notes: Used only when a recurrent-type KVCache policy is selected and a custom retained length is required. `n_keep_aligned` should be aligned according to the model's `kvcache_group_size`. For models with a system prompt, Runtime normally retains the system-prompt portion automatically.

4.5.2 RKNN3KVCachePolicyParam

KVCache-policy parameters:

Name	Description
`recurrent`	Recurrent parameters (`RECURRENT`)
`save_checkpoint`	Checkpoint-save parameters (`SAVE_CHECKPOINT`), only effective for models containing linear attention or sliding attention
`reserved`	Reserved (`uint8[64]`)

SAVE_CHECKPOINT substructure fields:

Name	Description
`checkpoint_start_pos`	Position at which checkpoint saving begins (`int64`), must be aligned to 128; Runtime forcibly adjusts unaligned values
`checkpoint_interval`	Token interval between saved checkpoints (`int64`), must be aligned to 128; Runtime forcibly adjusts unaligned values
`max_checkpoint_count`	Maximum number of saved checkpoints (`int64`), should be less than `(max_context_len - checkpoint_start_pos) / checkpoint_interval`; if exceeded, Runtime adjusts it to the maximum allowed value
`checkpoint_tail_overwrite`	Whether to use the last checkpoint slot as a rolling overwrite slot (`bool`). When True, the first `max_checkpoint_count - 1` slots remain fixed, while subsequent out-of-range checkpoints continuously overwrite the last slot

4.5.3 RKNN3KVCacheClearPolicyParam

KVCache-clear policy parameters:

Name	Type	Description
`keep_n_tokens`	`RKNN3KVCacheClearPolicyParamKeepNTokens`	Parameters for `RKNN3_KVCACHE_KEEP_N_TOKENS`
`reserved`	`uint8[64]`	Reserved field

`keep_n_tokens` substructure fields:

Name	Type	Description
`n_tokens`	`uint32`	Number of tokens to retain when clearing KVCache

Example:

```
clear_param = RKNN3KVCacheClearPolicyParam()
clear_param.keep_n_tokens.n_tokens = 128
rknn_lite.clear_kvcache(
    RKNN3KVCacheClearPolicy.RKNN3_KVCACHE_KEEP_N_TOKENS,
    param=clear_param,
)
```

4.5.4 RKNN3AttentionKvCacheLens

Name	Description
`RKNN3AttentionKvCacheLens`	KV Cache buffer-length configuration for an attention type. Fields include `attention_type`, `n_kvcache_buffer_lens`, and `kvcache_buffer_lens`; `kvcache_buffer_lens` contains at most `RKNN3_MAX_KVCACHE_LEN_GROUPS` (16) entries.

4.5.5 RKNN3KvCacheLenGroupInfo

Name	Description
`RKNN3KvCacheLenGroupInfo`	Per-Core KV Cache length-group information used by `RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO`. Fields include `core_id`, `n_groups`, `active_group_id`, and `kvcache_sizes`. `n_groups = 0` indicates compatibility with the old single-group mode. `kvcache_sizes[group_id]` gives the KV Cache size for that group on the corresponding Core.

Usage Notes: Returned by `RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO`. `n_groups` gives the number of selectable length groups, `active_group_id` identifies the current group, and `kvcache_sizes[group_id]` gives the cache size for the corresponding Core in that group. This information can be used for validity checks before calling `set_kvcache_group()`.

4.5.6 RKNN3AttentionConfig

Attention configuration describing the KVCache, positional encoding, and multimodal positional encoding (mRoPE) parameters of one attention branch in a model:

Field	Type	Description
`mask_name`	`char[256]`	Mask name
`kvcache_buffer_len`	`int32`	KVCache buffer length
`kvcache_realtime`	`bool`	Whether KVCache is real-time
`kvcache_dtype`	`int`	KVCache data type (`RKNN3KVCacheDtype`)
`kvcache_store_method`	`int`	KVCache storage method (`RKNN3KVCacheStoreMethod`)
`kvcache_group_size`	`int32`	KVCache group size
`kvcache_residual_depth`	`int32`	KVCache residual depth
`sliding_window_size`	`int32`	Sliding-window size
`attention_type`	`int`	Attention type (`RKNN3AttentionType`)
`position_name`	`char[256]`	Positional-encoding name
`max_position_embeddings`	`int32`	Maximum positional-encoding range
`position_embeddings_host_storage`	`bool`	Whether positional encodings are stored on the Host
`mrope_type`	`char[256]`	mRoPE type
`mrope_section`	`int32[8]`	mRoPE section
`n_mrope_section`	`int32`	Number of valid mRoPE section elements
`mrope_new_id_name`	`char[256]`	New mRoPE ID name
`kvcache_buffer_lens`	`int32[16]`	List of KVCache buffer lengths
`n_kvcache_buffer_lens`	`int32`	Number of valid entries in `kvcache_buffer_lens`

Field	Type	Description
`tokens_per_page`	`uint32`	Number of tokens held by each page
`rollback_tokens_align`	`uint32`	Alignment granularity for token rollback

Usage Notes: Returned by `RKNN3_QUERY_ATTENTION_CONFIG`; the array length is taken from `n_attention_configs` in `RKNN3_QUERY_LLM_CONFIG`. For paged KV Cache, `tokens_per_page` converts a context length into the required page count. Entries with `attention_type` equal to `RKNN3_ATTENTION_TYPE_FULL_ATTENTION` are the primary source of paged-cache configuration.

4.5.7 RKNN3CachePageAttr

Page attributes for a single cache type:

Field	Type	Description
`cache_type`	`int32`	Cache type (`RKNN3PageCacheType`)
`page_size`	`uint64`	Size of one page in bytes
`max_page_num`	`uint32`	Maximum page capacity fixed at compile time; does not change with binding or growth
`reserved`	`uint8[32]`	Reserved field

Usage Notes: Provided by `RKNN3KvCachePageInfo.caches`. When allocating pages, use `page_size` as the byte size passed to `create_mem()` and `max_page_num` as the maximum number of pages that can be bound for that cache type.

4.5.8 RKNN3KvCachePageInfo

Model-side geometry information for paged KV Cache, used with `RKNN3_QUERY_KVCACHE_PAGE_INFO`:

Field	Type	Description
`core_id`	`int32`	Input parameter specifying a physical Core ID
`max_request_num`	`uint32`	Maximum concurrent request count supported by continuous batching
`n_cache_types`	`uint32`	Number of valid cache types; 0 for non-paged models
`caches`	`RKNN3CachePageAttr[16]`	Per-type page attributes
`reserved`	`uint8[128]`	Reserved for future extensions

Usage Notes: Set `core_id` before querying; it can be obtained from `RKNN3AllocationInfo.core_id`, returned by `RKNN3_QUERY_ALLOCATION_INFO`. If `n_cache_types` is 0, the model does not use paged KV Cache; use `set_kvcache_mem()` instead.

4.5.9 RKNN3KvCachePageBinding

Binding entry for paged KV Cache, passed to `set_kvcache_pages()`:

Field	Type	Description
`core_id`	`int32`	Physical Core ID
`cache_type`	`int32`	Cache type (`RKNN3PageCacheType`)
`request_index`	`uint32`	Request index
`n_pages`	`uint32`	Number of pages (prefix length); 0 unbinds this entry
`pages`	`RKNN3TensorMemory**`	Array of page pointers, each pointing to a `page_size`-byte allocation on `core_id`
`reserved`	`uint8[64]`	Reserved field

Usage Notes: The caller owns the pointer array referenced by `pages` and must ensure it is not reclaimed during the `set_kvcache_pages()` call or while the binding remains valid.

4.6 LoRA Structure

LoRA-related Python APIs use `RKNN3Lora` as the adapter description object. This object is both returned by `query_lora()` and used as an argument for load/enable/disable/unload operations.

4.6.1 RKNN3Lora

LoRA configuration:

Field	Type	Description
`lora_name`	`char[256]`	LoRA name
`scale`	`float`	Scale factor

Usage Notes:

- `lora_name` identifies the adapter. Normally use the name returned by `query_lora()` directly.
- `scale` controls the strength of LoRA for the current inference. This field can be modified before enabling, then applied to the specified Session through `enable_lora()`.
- The same `RKNN3Lora` can be enabled/disabled independently on different Sessions.

4.7 Image, Audio, and Video Structures

`RKNN3Image`, `RKNN3Audio`, and `RKNN3Video` describe multimodal embeddings that have already been extracted, together with their corresponding special tokens. They do not represent raw media files themselves. When the vision model and LLM are decoupled, the application first runs the vision model and then places its output embedding into these structures.

4.7.1 RKNN3Image

Image input:

Field	Type	Description
`image_embed`	`Float16*`	Image embedding
`n_image_tokens`	`uint64`	Number of image tokens
`n_image`	`uint64`	Number of images
`image_start`	`char*`	Image-start token
`image_end`	`char*`	Image-end token
`image_content`	`char*`	Image-content token
`image_width`	`uint64`	Image width
`image_height`	`uint64`	Image height

Usage Notes: A typical VLM construction is:

```
image = RKNN3Image()
image.image_embed = vision_output.ctypes.data_as(ctypes.POINTER(Float16))
image.n_image_tokens = vision_output.shape[1]
image.n_image = vision_output.shape[0]
image.image_width = image_size
image.image_height = image_size
image.image_start = b"<|vision_start|>"
image.image_end = b"<|vision_end|>"
image.image_content = b"<|image_pad|>"
```

`image_start/image_end/image_content` must match the multimodal special tokens of the specific model's Tokenizer/Chat Template. The NumPy buffer referenced by `image_embed` must remain valid until `session_run()` completes.

4.7.2 RKNN3Audio

Audio input:

Field	Type	Description
`audio_embed`	`Float16*`	Audio embedding
`n_audio_tokens`	`uint64`	Number of audio tokens
`n_audio`	`uint64`	Number of audio segments
`audio_start`	`char*`	Audio-start token
`audio_end`	`char*`	Audio-end token
`audio_content`	`char*`	Audio-content token

Usage Notes: The field organization is similar to `RKNN3Image`. The key data are the audio embedding, token count for each audio segment, number of audio segments, and the model-defined start/end/content special tokens. The input is a feature embedding, not a PCM/WAV file path.

4.7.3 RKNN3Video

Video input:

Field	Type	Description
`video_embed`	`Float16*`	Video embedding
`n_frame_tokens`	`uint64`	Number of frame tokens
`n_frame_per_video`	`uint64`	Number of frames per video
`n_video`	`uint64`	Number of videos
`video_start`	`char*`	Video-start token
`video_end`	`char*`	Video-end token
`video_content`	`char*`	Video-content token
`frame_width`	`uint64`	Frame width
`frame_height`	`uint64`	Frame height

Usage Notes: Used to pass frame-level embeddings for one or more videos. `n_frame_tokens`, `n_frame_per_video`, and `n_video` jointly describe the logical organization of the embedding. Special tokens must likewise match model conventions.

4.7.4 RKNN3ImRect

Name	Description
`RKNN3ImRect`	Image rectangular region. Fields include `x`, `y`, `width`, and `height`.

4.7.5 RKNN3ImMetadata

Name	Description
`RKNN3ImMetadata`	Image metadata. Fields include `peer_im_mem_addr`.

4.7.6 RKNN3ImMem

Name	Description
`RKNN3ImMem`	Image memory. Fields include `width`, `height`, `stride`, `format`, `sync_to_host`, `data_mem`, and `metadata`.

4.8 Callback Types

Python can construct callbacks through `ctypes.CFUNCTYPE` and assign them to `RKLLMCallback`. Because the low-level Runtime invokes these Python objects during subsequent inference, callback function objects, userdata, and referenced Tokenizer/Embedding data must remain valid throughout the Session lifetime and must not be garbage-collected by Python.

4.8.1 LLMResultCallback

LLM result callback function type used to process results generated by LLM inference.

```
typedef int (*LLMResultCallback)(void* userdata, RKLLMResult* result, LLMCallState
state)
```

Parameters: | Name | Description | | --- | --- | | `userdata` | User-defined data pointer passed when the callback is registered | | `result` | Pointer to the LLM inference result (`RKLLMResult`), containing generated token IDs and count | | `state` | LLM call state (`LLMCallState`), such as running, finished, or error |

Return value: Return 0 on success; non-zero on failure.

Usage Notes: For streaming output, it is not recommended to produce final text directly from each callback's individual token. Instead, accumulate tokens and decode them together, and flush remaining text on FINISH/STOP/MAX_NEW_TOKEN_REACHED. This avoids the replacement

character `` when a UTF-8 multibyte character is split across tokens. The Callback can also call `session_stop()` based on business conditions.

4.8.2 LLMSamplingCallback

Sampling callback function type used to customize the sampling strategy during LLM inference, process logits, and return the selected token ID.

```
typedef int (*LLMSamplingCallback)(void* userdata, Float16* logits, char*
logits_name)
```

Parameters: | Name | Description | | --- | --- | | `userdata` | User-defined data pointer passed when the callback is registered | | `logits` | Pointer to the logits array (`Float16` type) | | `logits_name` | Logits-name string |

Return value: | Name | Description | | --- | --- | | `> 0` | Selected token ID | | `< 0` | Error code |

4.8.3 LLMGetEmbedCallback

Embedding callback function type used to obtain the embedding vector corresponding to specified tokens.

```
typedef int (*LLMGetEmbedCallback)(void* userdata, int32_t* tokens, uint64_t
num_tokens, void* embed, uint64_t len)
```

Parameters:

Name	Description
`userdata`	User-defined data pointer passed when the callback is registered
`tokens`	Token-ID array
`num_tokens`	Number of tokens in the token array
`embed`	Pointer to the buffer used to store embedding output
`len`	Length of the embedding buffer in bytes

Return value: Return 0 on success; non-zero on failure.

Usage Notes: When implementing this callback, first confirm that the target buffer length matches `num_tokens * embedding_dim * sizeof(float16)`, then look up embeddings from an FP16 embedding table shaped `[vocab_size, embedding_dim]` according to token IDs and write them into the Runtime-provided target buffer. Token IDs must be within the valid vocabulary range.

4.8.4 LLMTokenizerCallback

Tokenizer callback function type used to convert input text into a sequence of token IDs.

```
typedef int (*LLMTokenizerCallback)(void* userdata, const char* text, int32_t
text_len, int32_t* tokens, int32_t n_tokens_max)
```

Parameters: | Name | Description | | --- | --- | | `userdata` | User-defined data pointer passed when the callback is registered | | `text` | Pointer to the input text string | | `text_len` | Length of the input text | | `tokens` | Pointer to the token-ID array used to store tokenization results | | `n_tokens_max` | Maximum number of tokens that can be generated |

Return value: | Name | Description | | --- | --- | | `> 0` | Actual number of generated tokens | | `< 0` | Error code |

Usage Notes: The callback converts `text/text_len` into token IDs and writes at most `n_tokens_max` elements. The application can use HuggingFace `AutoTokenizer` or a custom Tokenizer. Return the actual number of written tokens; a negative value indicates failure.

4.8.5 LLMOutputCallback

Output callback function type used to obtain model output Tensors during LLM inference.

```
typedef int (*LLMOutputCallback)(void* userdata, RKNN3Tensor* output_tensors,
uint32_t n_output_tensors, LLMOutputCallbackState state)
```

Parameters: | Name | Description | | --- | --- | | `userdata` | User-defined data pointer passed when the callback is registered | | `output_tensors` | Pointer to the output-Tensor array (`RKNN3Tensor`) | | `n_output_tensors` | Number of output Tensors | | `state` | Output-callback state (`LLMOutputCallbackState`), such as Prefill processing, Prefill finished, Decode processing, or Decode finished |

Return value: Return 0 on success; non-zero on failure.

Usage Notes:

- Prepare `RKNN3Tensor` objects and their memory for the outputs to observe, and assign them to `RKLLMCallback.output_tensors/n_output_tensors`;
- In the callback, use `tensor.attr` to read output metadata and `tensor.mem.virt_addr` to read actual data;
- `state` distinguishes Prefill and Decode stages. If the application only needs results from a particular stage, check `state` before reading output, for example reading Prefill embedding only at `PREFILL_FINISHED`;
- If Output Callback memory is created explicitly by the application, call `destroy_mem()` for each buffer before exit.

4.8.6 LLMInputCallback

Input callback function type:

```
int (*LLMInputCallback)(void* userdata, RKNN3Tensor* tensors, uint32_t n_tensors,
LLMInputCallbackParam param)
```

This callback allows the user to fill or update specified input Tensors during LLM inference and is normally used together with `input_tensors_index` and `n_input_tensors`.

Usage Notes:

- `input_tensors_index` specifies model input indices that are filled by the application, and `n_input_tensors` specifies the number of indices;
- When the callback is triggered, `tensors` contains only the inputs that need application handling for the current callback. `tensor.attr` provides input name, Shape, and dtype, while `tensor.mem` provides the target buffer;
- `param.tokens`, `param.num_tokens`, and `param.pos` can be used to generate inputs dynamically according to current tokens and context position;
- This mechanism is suitable for models requiring additional inputs updated by token/position during inference, such as per-layer embeddings or Host RoPE Cache;
- Specific extended-input names, data layout, and filling algorithms are model conventions. The application should process them according to model export results and input attributes.

4.8.7 RKLLMCallback

LLM callback structure:

Field	Type	Description
`result_callback`	`LLMResultCallback`	Result callback
`result_userdata`	`void*`	Result userdata
`sampling_callback`	`LLMSamplingCallback`	Sampling callback
`sampling_userdata`	`void*`	Sampling userdata
`tokenizer_callback`	`LLMTokenizerCallback`	Tokenizer callback
`tokenizer_userdata`	`void*`	Tokenizer userdata
`embed_callback`	`LLMGetEmbedCallback`	Embedding callback
`embed_userdata`	`void*`	Embedding userdata
`output_callback`	`LLMOutputCallback`	Output callback
`output_userdata`	`void*`	Output userdata

Field	Type	Description
`output_tensors`	`RKNN3Tensor*`	Output Tensors
`n_output_tensors`	`uint32`	Number of output Tensors
`input_callback`	`LLMInputCallback`	Input callback
`input_userdata`	`void*`	Input-callback userdata
`input_tensors_index`	`int32*`	Input-Tensor indices
`n_input_tensors`	`uint32`	Number of input Tensors

Usage Notes:

- A standard LLM scenario normally registers at least `result_callback`, `tokenizer_callback`, and `embed_callback`;
- Input Callback requires `input_callback/input_userdata` plus `input_tensors_index/n_input_tensors`;
- Output Callback requires `output_callback/output_userdata` plus `output_tensors/n_output_tensors`;
- `ctypes` Callback objects, `ctypes.py_object` userdata, input-index arrays, output-Tensor arrays, and NumPy/mmap objects referenced by callbacks must all retain Python references throughout Session and callback use;
- In both synchronous and asynchronous Session scenarios, do not rely on local variables simply remaining unreclaimed. A safer approach is to store these objects in long-lived variables, object members, or a dedicated keepalive list.

4.8.8 LLMInputCallbackParam

LLM input-callback parameters describing token, Shape, and context-position information for the current Input Callback.

Name	Description
`tokens`	Token-ID array corresponding to the current callback
`num_tokens`	Number of tokens to process in this callback
`shape_id`	Shape ID used by the current execution
`pos`	Starting position of the current tokens within the current context
`mrope_pos`	Information related to M-RoPE position
`mrope_start`	M-RoPE starting position
`system_prompt_seqlen`	System-Prompt sequence length
`valid_system_prompt_seqlen`	Currently valid system-Prompt sequence length
`max_position_embeddings`	Maximum positional-encoding length supported by the model
`reserved`	Reserved field

Usage Notes:

- For extended inputs that only need token-based lookup, mainly use `tokens` and `num_tokens`;
- For data that needs to be sliced according to context position, use `pos` to calculate the data region to fill;
- Whether M-RoPE and system-Prompt-related fields are required depends on the specific model;
- Callback parameters are provided by Runtime. The application should only read their values and must not retain Runtime pointers from them for access after the callback ends.

4.9 Wrapper Classes

Wrapper classes convert Python-side objects such as `numpy.ndarray` into auxiliary input descriptions recognized by RKNN3Lite high-level interfaces. DeepStack VLM is a typical use case.

4.9.1 RKNN3AuxTensorWrapper

Auxiliary-Tensor wrapper class:

Field	Type	Description
`aux_data`	`numpy array`	Auxiliary data
`index`	`int`	Index
`align_size`	`int`	Alignment size

Usage Notes: In a DeepStack VLM scenario, additional outputs from the vision model can be wrapped as AUX input:

```
aux = RKNN3AuxTensorWrapper()
aux.index = llm_input_index
aux.aux_data = vision_outputs[i]
aux.align_size = aligned_size
inputs.append(aux)
```

`index` should correspond to the actual AUX input position in the LLM model. It is recommended to confirm this through Tensor-attribute queries rather than assuming a fixed index. `align_size` should match the alignment requirement of that input.

5. Public Utility Functions

5.1 rknn_dtype_to_numpy_dtype

API	`rknn_dtype_to_numpy_dtype`
Function	Converts an RKNN3 Tensor data-type enumeration value into the corresponding NumPy data type. This function is used when processing Tensor data in Python to ensure correct type mapping.
Parameters	`dtype_enum`: RKNN3 Tensor-type enumeration value (`RKNN3TensorType`).
Return value	Corresponding NumPy data type (`numpy.dtype`). If the input enumeration value has no mapping, returns `numpy.float32`.

Example:

```
from rknn3lite.api.rknn3_types import rknn_dtype_to_numpy_dtype, RKNN3TensorType

# Convert FP16 type
np_dtype = rknn_dtype_to_numpy_dtype(RKNN3TensorType.RKNN3_TENSOR_FLOAT16)
print(np_dtype)  # Output: <class 'numpy.float16'>
```

5.2 rknn3_get_layout_string

API	`rknn3_get_layout_string`
Function	Gets the corresponding layout string from a Tensor-layout enumeration value. This function is intended for debugging and log output to help users understand Tensor layout formats.
Parameters	`layout`: RKNN3 Tensor-layout enumeration value (`RKNN3TensorLayout`).
Return value	Layout string such as `"NCHW"` or `"NHWC"`. If the enumeration value is invalid, returns `"UNKNOWN"`.

Example:

```
from rknn3lite.api.rknn3_types import rknn3_get_layout_string, RKNN3TensorLayout

# Get NHWC layout string
layout_str = rknn3_get_layout_string(RKNN3TensorLayout.RKNN3_TENSOR_NHWC)
print(layout_str)  # Output: NHWC
```

5.3 rknn3_get_type_string

API	`rknn3_get_type_string`
Function	Gets the corresponding type string from a Tensor data-type enumeration value. This function is intended for debugging and log output to help users understand Tensor data types.
Parameters	`dtype`: RKNN3 Tensor data-type enumeration value (`RKNN3TensorType`).
Return value	Data-type string such as `"FP32"` or `"INT8"`. If the enumeration value is invalid, returns `"UNKNOWN"`.

Example:

```
from rknn3lite.api.rknn3_types import rknn3_get_type_string, RKNN3TensorType

# Get INT8 type string
type_str = rknn3_get_type_string(RKNN3TensorType.RKNN3_TENSOR_INT8)
print(type_str)  # Output: INT8
```

5.4 rknn3_get_qnt_type_string

API	`rknn3_get_qnt_type_string`
Function	Gets the corresponding quantization-type string from a Tensor quantization-type enumeration value, such as `NONE`, `PER_LAYER_SYMMETRIC`, or `PER_CHANNEL_ASYMMETRIC`. If the enumeration value is invalid, returns `UNKNOWN`.
Parameters	`qnt_type`: RKNN3 Tensor quantization-type enumeration value (`RKNN3TensorQntType`).
Return value	Quantization-type string.

5.5 dump_tensor_attr

API	`dump_tensor_attr`
Function	Prints detailed Tensor attributes to the console. This function is intended for debugging and helps developers inspect Tensor information such as Shape, data type, and Layout.
Parameters	`attr`: RKNN3 Tensor-attribute structure (`RKNN3TensorAttr`).
	`prefix`: Print-prefix string. Default: `"tensor"`.
Return value	None

Example:

```
from rknn3lite.api.rknn3_types import dump_tensor_attr

# Assume a Tensor attribute object attr exists
dump_tensor_attr(attr, prefix="input")
# Example output:
# input_0: name=input_tensor, n_dims=4, shape=[1, 3, 224, 224], layout=NHWC,
dtype=FP32, ...
```

5.6 get_os_platform

API	`get_os_platform`
Function	Gets the current operating-system platform string. This function is used to determine the runtime architecture (32-bit or 64-bit) and helps with platform-specific configuration and compatibility checks.
Parameters	None
Return value	Platform string in the form `"{OS}_{architecture}"`, such as `"Linux_x64"` or `"Windows_x32"`.

Example:

```
from rknn3lite.api.rknn3_types import get_os_platform

platform = get_os_platform()
print(platform)  # Output: Linux_x64 (on a 64-bit Linux system)
```

6. Exceptions and Error Handling

RKNN3 Toolkit Lite Python APIs mainly report runtime errors through **return values and logs**. Some interfaces return `-1` when parameter validation or a low-level Runtime call fails, while query/object interfaces normally return `None`. The specific cause of a low-level RKNN3 Runtime error can be determined with reference to 4.1.2 Status Codes.

In addition to Runtime errors, Python parameter processing, file access, NumPy, Tokenizer, `ctypes` Callbacks, and user-defined logic may still raise Python exceptions. Applications should check both API return values and logs and should not rely solely on `try/except` to determine whether RKNN3 Runtime execution succeeded.

6.1 Error Returns and Logs

RKNN3 Toolkit Lite does not require all APIs to use the same failure-return form. Common cases are as follows:

API Type	Success Return	Common Failure Return
Status API	`0`	`-1` or a low-level non-zero return value
Query/object API	Query result or object	`None`
List API	List	`None` or an empty result
LoRA query	`(lora_list, n_lora)`	`(None, 0)`
Context duplication	`(ret, new_context)`	`(-1, None)`
LLM inference	`(ret, stats)`	Usually `(-1, [])`
Image-memory creation	`(ret, im_mem)`	Failure result follows the API return value

Therefore, callers should determine failure according to the "Return value" description of each API in Chapter 3. For example:

```
ret = rknn.init_runtime(target="rk1820", core_mask=0x01)
if ret != 0:
    print(f"init_runtime failed, ret={ret}")
    rknn.release()
    return
```

Query APIs should check for `None`:

```
io_num = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_IN_OUT_NUM)
if io_num is None:
    print("query input/output number failed")
    rknn.release()
    return
```

For LLM inference, `session_run()` / `session_run_async()` normally return `(-1, [])` when not initialized or when an exception is caught. Check the return code directly:

```
ret, stats = rknn.session_run(prompt="hello")
if ret != 0:
    print(f"session_run failed, ret={ret}")
```

Creating `RKNN3Lite(verbose=True)` enables more detailed runtime logs. Logs can also be saved to a file through `verbose_file`:

```
rknn = RKNN3Lite(
    llm_mode=True,
    verbose=True,
    verbose_file="./rknn3lite.log"
)
```

When a problem occurs, it is recommended to save both Python-side logs and low-level Runtime return codes so that the failure can be identified as parameter validation, Python-wrapper logic, or device Runtime.

6.2 Runtime and Model Initialization Errors

Runtime and model initialization are prerequisites for all subsequent inference operations. Common problems include:

Symptom	Possible Cause	Handling Method
`load_rknn()` returns failure	Model path or weight path does not exist or is invalid	Check that the `.rknn` and `.weight` file paths exist and are valid
`init_runtime()` returns failure	Possible causes include model/weight mismatch, invalid model format, or platform mismatch	Check that the model and weight are a matching pair and that the model format matches the target platform
`init_runtime()` returns failure	`target`, `core_mask`, `device_id`, model, Runtime, or device state is abnormal	Check the parameters and use the status codes in Section 4.1.2 together with Runtime logs for diagnosis
`RKNN3_ERR_MODEL_INVALID`	Model file is invalid or cannot be loaded correctly by the current Runtime	Confirm that the model file is complete and that the model matches the current SDK/Runtime
`RKNN3_ERR_TARGET_PLATFORM_UNMATCH`	Model target platform does not match the actual runtime platform	Use an RKNN model that matches the target NPU platform
`RKNN3_ERR_DEVICE_UNAVAILABLE`	Target device is unavailable	Use `get_devices_id()` to check whether the device exists and inspect device-side services
`RKNN3_ERR_INCOMPATIBLE_VERSION`	Runtime-related component versions do not match	Confirm that the Python Wheel, Runtime shared library, and device-side components use compatible versions
`RKNN3_ERR_KEY_INVALID` / `RKNN3_ERR_DECRYPT_FAILED`	Decryption key is invalid, key does not match the model, or encrypted data is corrupted	Check `decrypt_key_path` / `decrypt_key_data` and the encrypted model file
`RKNN3_WARN_NPU_CORE_UNUSED`	Some specified NPU Cores are not used by model execution	This state is only a warning. If execution results are correct, execution may continue

`core_mask` is a common source of initialization configuration errors. It is recommended to confirm the Core count supported by the model and device before constructing the mask rather than assuming every model should use all NPU Cores.

If `init_runtime()` has failed, do not continue calling `inference()`, `rknn3_query()`, `init_llm_session()`, or other interfaces that depend on Runtime. Handle the initialization error first and then recreate or reinitialize Runtime.

6.3 Query, Input, and Inference Errors

Query and inference errors are usually related to call order or mismatches in input count, Shape, dtype, or Layout.

6.3.1 Query Failure

`rknn3_query()` normally returns `None` on failure. Common causes include:

- Runtime has not been initialized;
- The Query Cmd is not supported;
- The input/output index exceeds the actual model range;
- The current model does not support the queried item;
- The underlying Runtime Query returns an error.

It is recommended to query input/output count first and then query Tensor attributes using valid indices:

```
io_num = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_IN_OUT_NUM)
if io_num is None:
    return

for i in range(io_num.n_input):
    attr = rknn.rknn3_query(
        RKNN3QueryCmd.RKNN3_QUERY_INPUT_ATTR,
        index=i
    )
    if attr is None:
        print(f"query input attr[{i}] failed")
        return
```

6.3.2 General-Model Input Errors

Before `inference()`, confirm that:

- The number of `inputs` matches the model input count;
- NumPy Shape matches the model's current Shape;
- dtype can be correctly converted to the data type required by the model;
- `data_format` for 4-D input matches the actual logical layout;
- Dynamic-Shape input belongs to a Shape combination supported by the model.

When `RKNN3_ERR_INPUT_INVALID` occurs, first inspect the input information above. When `RKNN3_ERR_OUTPUT_INVALID` occurs, inspect output-Tensor configuration and explicit output memory.

For a dynamic-Shape model, use:

```
shape_infos = rknn.get_dynamic_shape_infos()
```

to inspect supported Shape combinations. When explicit switching is required, use:

```
ret = rknn.set_shape(shape_id)
```

6.4 LLM Session and Input Errors

In addition to Runtime, LLM/VLM APIs depend on an initialized LLM Session.

If Session, KVCache, LoRA, Chat Template, or similar APIs are called before the LLM Runtime or Session is initialized, the Python layer normally logs an error such as "LLM runtime environment is not inited" and returns `-1` or `None`.

The recommended order is:

```
RKNN3Lite(llm_mode=True)
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
init_llm_session()
    ↓
session_run()
```

6.4.1 Session Index Errors

Multi-Session usage requires continuous `session_index` values. When creating a new Session:

- `session_index < 0`: Invalid;
- `session_index < current Session count`: That index is already initialized;
- `session_index > current Session count`: Earlier Sessions have been skipped;
- The correct new Session index should equal the current Session count.

For example, if only Session 0 exists, Session 1 must be created next; Session 2 cannot be created directly.

When a Session-index error occurs, use:

```
session_count = rknn.get_session_count()
```

to get the current number of initialized Sessions and then determine the new `session_index`.

6.4.2 Mutually Exclusive LLM Main Inputs

For `session_run()` and `session_run_async()`:

- `prompt`
- `embeds`
- `tokens`

are mutually exclusive main inputs. Only one can be selected in a single call.

Incorrect:

```
rknn.session_run(
    prompt="hello",
    tokens=[1, 2, 3]
)
```

Correct:

```
rknn.session_run(prompt="hello")
```

or:

```
rknn.session_run(tokens=[1, 2, 3])
```

`inputs` is used for multimodal/auxiliary input such as `RKNN3Image`, `RKNN3Audio`, `RKNN3Video`, or AUX, and can be combined with one main-input form according to model requirements.

6.4.3 Context-Length Errors

LLM `max_context_len` must not exceed model `RKNN3LLMConfig.max_ctx_len`. It is recommended to query before creating a Session:

```
llm_config = rknn.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_LLM_CONFIG
)
```

and use:

```
llm_config.max_ctx_len
```

as the upper limit for application configuration.

During execution, `session_query_state()` can also be used to inspect:

- `n_total_tokens`
- `n_max_tokens`

When context approaches the limit, clear KVCache or end the current Session according to business requirements.

6.5 KVCache and LoRA Errors

6.5.1 KVCache

KVCache APIs can only be used on a valid LLM Session. Common problems include:

- Session is not initialized;
- `session_index` is invalid;
- KVCache file does not exist or does not match the current model/Session configuration;
- `kvcache_data` and `size` do not match when loading from memory;
- `group_id` exceeds the valid range returned by `RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO`;
- When external KVCache memory is used, the target group requires more memory than the supplied memory provides.

Before calling `set_kvcache_group()`, it is recommended to check:

```
infos = rknn.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO
)

if infos is not None:
    n_groups = infos[0].n_groups
    if 0 <= group_id < n_groups:
        rknn.set_kvcache_group(group_id)
```

Paged KV Cache uses `set_kvcache_pages()`. Common problems include:

- Calling this API for a non-paged model. First check that `n_cache_types` from `RKNN3_QUERY_KVCACHE_PAGE_INFO` is non-zero; otherwise use `set_kvcache_mem()`.
- Entries in `bindings` have a `core_id` or `cache_type` that does not match the model configuration, causing shape validation to fail.
- `n_pages` exceeds `RKNN3KvCachePageInfo.caches[i].max_page_num`.
- A page does not have the size specified by `RKNN3CachePageAttr.page_size`, or is not allocated on the Core specified by `core_id`.
- The pointer array referenced by `pages` is reclaimed during the call, or a page is released before it is unbound.
- The API is called while the Context is busy (for example, during inference).

Because `set_kvcache_pages()` is a Context-level operation, its error handling is independent of Sessions. Cross-check the results of `RKNN3_QUERY_KVCACHE_PAGE_INFO` and `RKNN3_QUERY_ATTENTION_CONFIG` when diagnosing configuration mismatches.

6.5.2 LoRA

LoRA should be used in the following order:

```
init_lora()
    ↓
query_lora()
    ↓
load_lora()
    ↓
enable_lora()
```

Common errors include:

- LLM Runtime has not been initialized;
- LoRA weight path or memory data is invalid;
- A nonexistent `lora_name` is used;
- `enable_lora()` is called before `load_lora()`;
- LoRA is operated on a released or nonexistent Session;
- LoRA does not match the Base model.

To confirm available adapters, prefer the `RKNN3Lora` information returned by `query_lora()` rather than hard-coding adapter names in the application.

6.6 Memory-Related Errors

Memory-related problems normally appear as memory-creation failure, synchronization failure, NPU execution failure, or abnormal output data.

6.6.1 Out of Memory

When the low-level Runtime returns:

```
RKNN3_ERR_OUT_OF_MEMORY
```

it means the current device or specified NPU Core cannot satisfy the memory requirement.

Use:

```
mem_info = rknn.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_DEVICE_MEM_INFO
)
```

to inspect total system memory, available memory, and memory information for each NPU Node.

For LLM, also pay attention to:

- `max_context_len`;
- KVCache length groups;
- User-defined KVCache memory;
- Number of simultaneous Sessions;
- Memory consumed by LoRA and additional Output Tensors.

6.6.2 Memory Synchronization Failure

`RKNN3_ERR_MEM_SYNC_FAILED` indicates that memory synchronization between CPU and device failed.

When using explicit Tensor Memory, note the following:

- Before CPU writes and NPU reads, use `RKNN3_MEMORY_SYNC_TO_DEVICE`;
- Before NPU writes and CPU reads, use `RKNN3_MEMORY_SYNC_FROM_DEVICE`;
- `mem_sync_range()` must satisfy `offset + length <= mem.size`;
- `length` and `offset` are measured in bytes;
- Memory that has already been passed to `destroy_mem()` must not be synchronized or accessed again.

6.6.3 External-Memory Lifetime

`create_mem_from_phys()`, `create_mem_from_fd()`, and user-provided Runtime memory depend on application-side buffers. The application must ensure that the underlying physical memory, FD, virtual address, and related resources remain valid while RKNN3 uses them.

`destroy_mem()` releases RKNN3's association object for that memory. Whether the external memory itself must also be released by the application depends on how it was originally allocated.

6.7 Callback Exceptions

LLM Callbacks run within the low-level Runtime call chain. An unhandled Python exception inside a Callback may cause the current callback or inference to fail, so it is recommended to catch exceptions inside the Callback.

Recommended form:

```python
def result_callback(userdata, result_ptr, state):
    try:
        # Process result
        return 0
    except Exception as e:
        print(f"result callback failed: {e}")
        return -1
```

Common Callback problems are shown below:

Problem	Cause	Handling Method
Callback stops triggering or an invalid access occurs	Callback Python object was garbage-collected	Retain references to Callback objects for the Session lifetime
userdata access error	Lifetime of `ctypes.py_object` or its pointer ended	Retain userdata and its underlying Python object
Embedding Callback returns failure	Buffer length or token ID is invalid	Check `num_tokens`, `embedding_dim`, `len`, and vocabulary range
Input Callback writes incorrect data	Tensor name, Shape, dtype, or current-position handling is wrong	Validate the target buffer through `tensor.attr` and `LLMInputCallbackParam`
Output Callback data is abnormal	Data was read at the wrong callback stage or dtype was interpreted incorrectly	Check `LLMOutputCallbackState` and interpret memory according to Tensor attr
Streaming text contains `�`	One token does not form a complete UTF-8 character	Accumulate tokens and decode together, or wait for later tokens

The following objects should remain valid at least until the Session no longer uses the corresponding Callback:

- `RKLLMCallback`;
- All `ctypes.CFUNCTYPE` Callback objects;
- `ctypes.py_object` userdata;
- `input_tensors_index`;
- `output_tensors`;
- NumPy, mmap, or other external buffers directly referenced by Callbacks.

6.8 Asynchronous-Execution Errors

General models and LLMs use different asynchronous flows.

General Models

The call order should be:

```
inference_async()
    ↓
wait()
    ↓
get_outputs()
```

Before an asynchronous task completes, do not release input, Runtime, or related explicit memory prematurely.

LLM

The return value of `session_run_async()` only means that the task has been submitted; it does not mean generation has completed. Actual completion should be determined through `LLMResultCallback`.

Common terminal states:

- `RKLLM_RUN_FINISH`
- `RKLLM_RUN_STOP`
- `RKLLM_RUN_MAX_NEW_TOKEN_REACHED`
- `RKLLM_RUN_ERROR`

To stop or pause a running request, call:

```
rknn.session_stop(session_index=0)
rknn.session_pause(session_index=0)
rknn.session_resume(session_index=0)
```

The application should coordinate running, stopping, pausing, resuming, and releasing operations on the same Session and avoid releasing a Session or Runtime while a task is still running.

6.9 Error-Diagnosis Recommendations

When an RKNN3 Toolkit Lite error occurs, it is recommended to troubleshoot in the following order:

1. **Confirm the call order**
   Check whether `load_rknn()` and `init_runtime()` have completed; for LLM, also confirm that `init_llm_session()` has completed.
2. **Check Python API return values**
   Determine failure according to the corresponding API contract, such as `ret != 0`, `result is None`, or another failure form.
3. **Inspect detailed logs**
   Use `verbose=True` and, when necessary, save complete logs through `verbose_file`.
4. **Confirm the low-level status code**
   Compare the Runtime error code in the logs with 4.1.2 Status Codes.
5. **Check the model and platform**
   Confirm that the RKNN model target platform, model file, Weight file, and current NPU platform match.
6. **Check versions**
   If `RKNN3_ERR_INCOMPATIBLE_VERSION` occurs, inspect the versions of the Python Wheel, Runtime shared libraries, and related device-side components.
7. **Check model I/O**
   Use `RKNN3_QUERY_IN_OUT_NUM`, `INPUT_ATTR`, and `OUTPUT_ATTR` to confirm input/output count, Shape, dtype, and Layout.
8. **Check device and memory**
   Use `get_devices_id()`, `RKNN3_QUERY_DEVICE_MEM_INFO`, and `RKNN3_QUERY_CORE_NUMBER` to confirm device, Core, and available memory.
9. **For LLM, further inspect Session state**
   Use `get_session_count()` and `session_query_state()` to inspect Session, Token, and KVCache state.
0. **Confirm resource lifetimes**
   Check whether Tensor Memory, NumPy Buffer, Callback, userdata, Session, or Runtime has been released prematurely while still in use.

If an error occurs in the low-level Runtime and Python logs cannot provide further diagnosis, preserve the following information for problem analysis:

- RKNN3 Toolkit Lite / Python Wheel version;
- Information returned by `get_sdk_version()`;
- Hardware platform and `device_id`;

- RKNN model and Weight-file information;
- `target` and `core_mask`;
- Complete error logs;
- Runtime status code;
- Minimal call flow that can reproduce the issue.